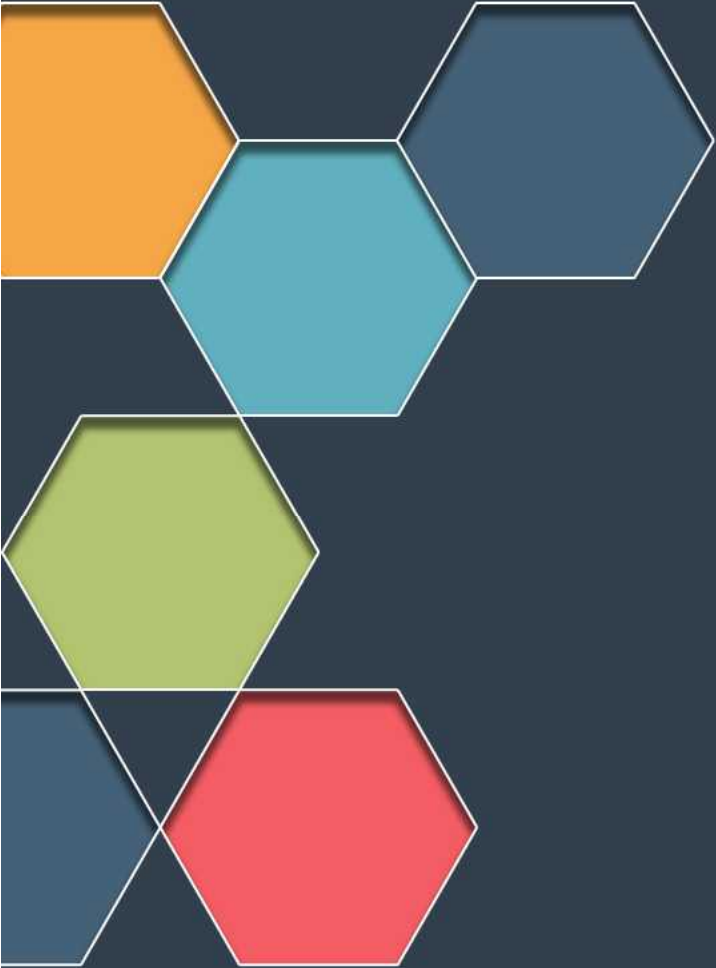




제5장 지식표현

학습 목표

2

- 여러 가지 지식 표현 방법을 살펴본다.
- 명제논리를 이해한다.
- 술어논리를 이해한다.
- 논리융합을 이해한다.
- 술어논리에서 사용되는 추론 기법을 이해한다.

1. 지식 표현

❖ 데이터 피라미드

- **데이터** (data)
 - 특정 분야에서 **관측된 아직 가공되지 않은 것**
 - 사실인 것처럼 관측되지만 **오류나 잡음** 포함 가능
- **정보** (information)
 - 데이터를 **가공**하여 어떤 **목적이나 의미**를 갖도록 한 것
- **지식** (knowledge)
 - 정보를 **취합**하고 **분석**하여 얻은 대상에 대해 사람이 **이해**한 것
- **지혜** (wisdom)
 - 경험과 학습을 통해서 얻은 지식보다 높은 수준의 **통찰**



지식 표현

❖ 지식(知識, knowledge)

- **경험이나 교육**을 통해 얻어진 **전문적인 이해**(understanding)와 체계화된 **문제 해결 능력**
- 어떤 주제나 분야에 대한 **이론적** 또는 **실제적인 이해**, 또는 현재 알려진 **사실과 정보**의 모음
- **암묵지**(暗黙知, tacit knowledge)
 - 형식을 갖추어 표현하기 어려운, 학습과 경험을 통해 쌓은 지식
- **형식지**(形式知, explicit knowledge)
 - 비교적 쉽게 형식을 갖추어 표현될 수 있는 지식
- **절차적 지식**(procedural knowledge)
 - 문제해결의 절차 기술
- **선언적 지식**(declarative knowledge)
 - 어떤 대상의 성질, 특성이나 관계 서술
- **컴퓨터를 통한 지식 표현 및 처리**
 - 프로그램이 쉽게 처리할 수 있도록 **정형화된 형태로 표현**
 - 규칙, 프레임, 논리, 의미망, 스크립트, 수치적 함수 등

지식표현 방법

5

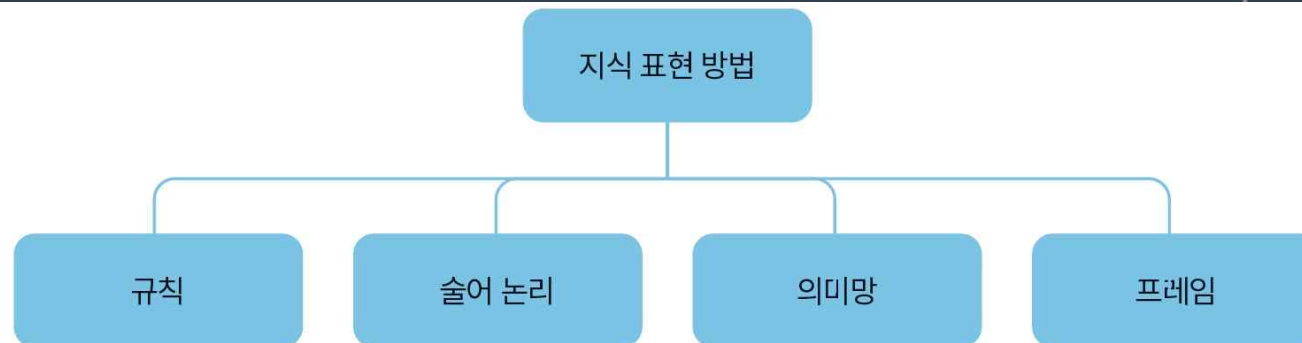


그림 5-1 지식 표현 방법의 종류

표 5-1 지식 표현 모델의 분류

지식 표현 모델	종류
생성 규칙(Production Rule) 또는 규칙	절차적(procedural)
술어 논리(Predicate Logic)	선언적(declarative)
의미망(Semantic Net)	선언적(declarative)
프레임(frame)	선언적(declarative)

규칙

- 규칙은 전제(IF문)와 결론(THEN)의 두 부분으로 구성된다. 규칙의 전제 조건이 일치하는 경우, 규칙은 점화되고 결론 부분이 실행된다.
- 규칙을 사용하는 시스템에는 작업 메모리라고도 하는 데이터베이스가 포함된다. 작업 메모리에는 현재 관측된 사실이나 상태, 지식이 저장된다

규칙 #1: IF 비가 온다.

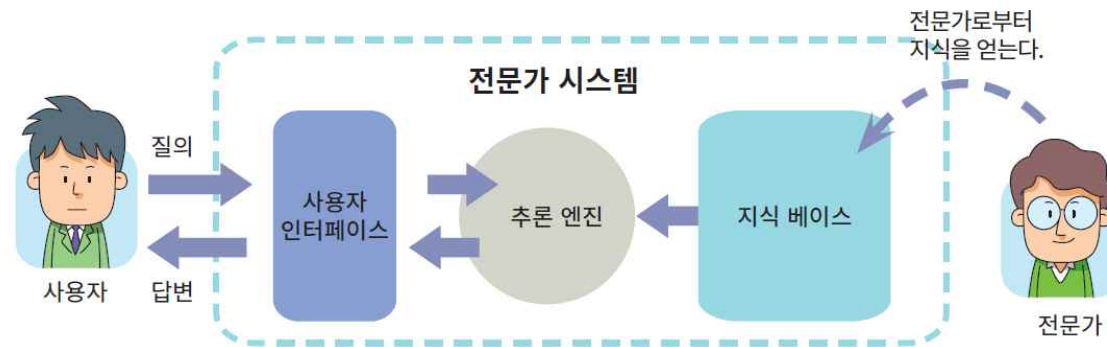
THEN 우산을 가져간다.

규칙 #2: IF 프로그램에 버그가 없다.

THEN 프로그램은 올바르게 동작한다.

규칙 #3: IF 습도가 높다 OR 온도가 30도 이상이다.

THEN 에어컨을 가동한다.



03 의미망

7

- 의미망(semantic network)은 방향 그래프를 이용하여 개념 간의 관계를 나타내는 방법이다.
- 그래프는 노드와 간선으로 이루어진다.
 - 노드는 사물(objects), 개념(concepts)등을 표현한다.
 - 간선(edge)은 사물이나 개념 사이의 관계이다.

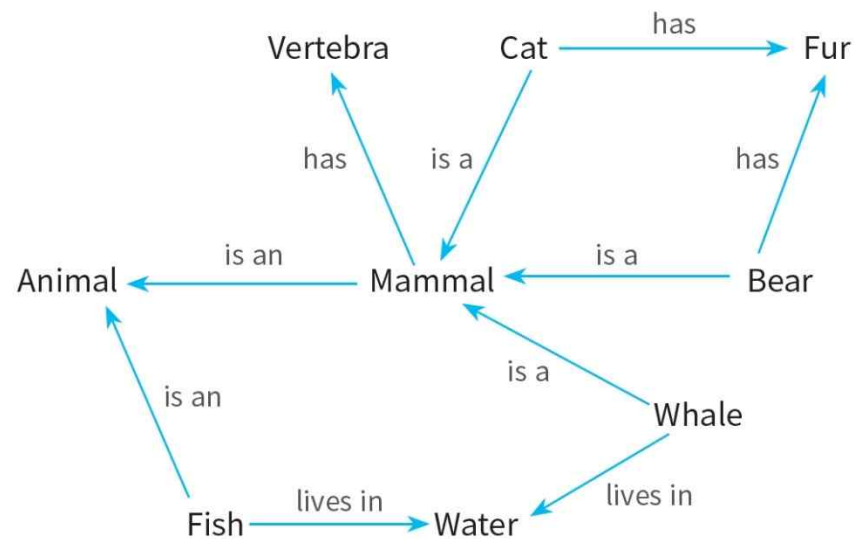


그림 5-3 의미망(출처: Public Domain, <https://commons.wikimedia.org/w/index.php?curid=1353062>)

의미망

❖ 의미망(semantic network)

- 지식을 **이항 관계**(binary relation)의 집합으로 표현
- 노드와 방향성 간선으로 구성되는 **그래프**를 사용해 지식 표현
 - **노드 (node)**
 - 대상, 개념, 행위, 상태, 사건
 - **간선 (edge)**
 - 관계가 있는 노드를 연결
 - 관계에 따른 **방향**
 - 관계의 의미를 나타내는 **라벨** 부여

의미망

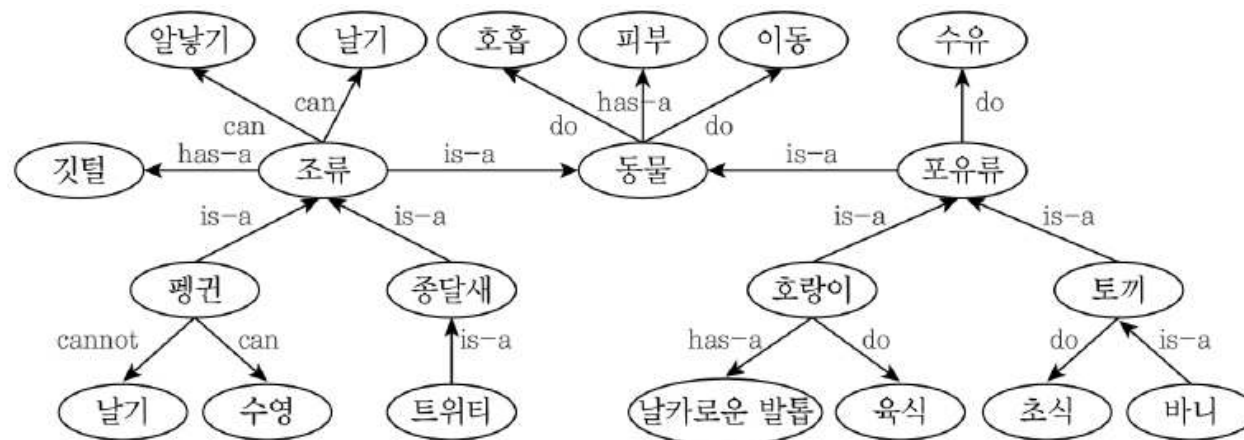
- 노드 사이의 간선은 “..의 일종이다”(is-a), “...을 가지고 있다”(has), “...은 하나의 예이다.” (inst-of)와 같은 관계를 나타낸다.
- 의미망은 매우 복잡한 개념이나 인과 관계를 잘 표현할 수 있지만 지식의 양이 커지면 너무 복잡해져서 조작이 어렵다는 단점도 가지고 있다.
- 의미망을 위한 표준 지침이 없기 때문에 시스템에 따라 의미망의 형태가 다를 수 있다.

의미망

❖ 의미망의 예

is-a(조류, 동물),
has-a(조류, 깃털),
can(조류, 날기),
cannot(펭귄, 날기),
do(동물, 호흡),
do(동물, 이동),
do(호랑이, 육식),
do(토끼, 초식),
can(펭귄, 수영)
do(포유류, 수유)

is-a(포유류, 동물)
can(조류, 알 낳기)
is-a(펭귄, 조류)
is-a(종달새, 조류)
has-a(동물, 피부)
has-a(호랑이, 날카로운 발톱)
is-a(토끼, 포유류)
is-a(트위터, 종달새)
is-a(바니, 토끼)



의미망의 예

의미망

❖ 의미망에서 사용되는 관계(relationship)

▪ is-a

- 하위 클래스와 상위 클래스 관계(예, is-a(조류,동물)), 또는 객체와 클래스의 관계(예, is-a(트위티,종달새))를 나타내어 계층 관계 표현
- 상위 계층의 속성을 상속
- 추이적(transitive) 관계 만족

$$\text{is-a}(X,Y) \wedge \text{is-a}(Y,Z) \rightarrow \text{is-a}(X,Z).$$

$$\text{is-a}(\text{펭귄},\text{조류}) \wedge \text{is-a}(\text{조류},\text{동물}) \rightarrow \text{is-a}(\text{펭귄},\text{동물})$$

▪ has-a

- 전체-부분 관계
- part-of와 역관계
 - has-a(X,Y)이면 part-of(Y,X) 성립
- 추이적 관계 만족

$$\text{has-a}(X,Y) \wedge \text{has-a}(Y,Z) \rightarrow \text{has-a}(X,Z).$$

$$\text{part-of}(X,Y) \wedge \text{part-of}(Y,Z) \rightarrow \text{part-of}(X,Z).$$

04 프레임

12

- 프레임(frame)은 1970년대에 Marvin Minsky가 제안한 지식 표현 방법으로 의미망(semantic network)에서 파생
- 프레임은 특정 객체와 그 속성을 묶어서 하나로 조직화하는 방법이며 개념, 객체, 상황들을 기술하는데 유리하다.
- 프레임은 지식을 구조화하는 유연하고 모듈식인 방법을 제공하여, 대량의 구조화된 정보를 저장하고 처리하며 추론해야 하는 AI 시스템에서 매우 유용하게 사용된다.

표 5-2 책을 프레임으로 표현한 예

슬롯	값
publisher	인피니티북스
title	인공지능
author	홍길동
edition	초판
year	2019
pages	700

프레임

13

- 프레임은 객체의 속성과 값으로 이루어져 있다.
- 개개의 속성은 슬롯(slot)이라고 불리고 슬롯은 값을 가질 수가 있다.
- 프레임은 프로그래밍 언어의 구조체나 객체(object)와 유사하며 객체의 필드에 해당하는 것은 슬롯이다.

표 5-3 컴퓨터를 프레임으로 표현한 예

슬롯	값
name	컴퓨터
subclass	기계
types	default: desktop if-added: Procedure INCREMENT_COMPUTER
speed	default: fast if-needed: Procedure CALCULATE_SPEED

프레임

❖ 프레임(frame)

- 민스키(M. Minsky, 1927~2016)가 제안한 지식표현 방법
- 특정 객체 또는 개념에 대한 전형적인 지식을 **슬롯(slot)의 집합**으로 표현하는 것
- 예. 컴퓨터를 표현한 프레임

frame-name	Computer	
frame-type	Class	
CPU	default	Intel
	data-type	string
	require	Intel AMD ARM SPARC
OS	default	Windows
	data-type	string
memory	data-type	integer
warranty	default	3years
HDD	default	1TB
price	data-type	integer
stock	default	in-stock

(frame

(**frame-name** Computer)
(**frame-type** class)
(**CPU** (default Intel)
 (data-type string)
 (require (Intel AMD ARM SPARC)))
(**OS** (default Windows)
 (data-type string))
(**memory** (data-type integer))
(**warranty** (default 3years))
(**HDD** (default 1TB))
(**price** (data-type integer))
(**stock** (default in-stock)))

프레임

❖ 프레임의 구성요소

▪ 슬롯(slot)

- 객체의 **속성**(attribute)을 기술하는 것
- **슬롯 이름**(slot name)과 **슬롯 값**(slot value)으로 구성
 - 슬롯 이름: 속성 이름
 - 슬롯 값: 속성의 값
- **슬롯 값(slot value)**
 - 복수 개의 **패싯**(facet)과 **데몬**(demon, 또는 프로시저)으로 구성

프레임

❖ 프레임의 구성요소 – Cont.

▪ 패싯 (facet)

- '측면' 또는 '양상'을 의미
- 속성에 대한 추가적인 정보를 지정하기 위해 사용
- 패싯 이름과 패싯 값의 쌍으로 구성
- 패싯 이름
 - value : 속성값 (수, 문자열, 다른 프레임의 포인터 등)
 - data-type : 속성값의 자료형
 - default : 디폴트값(속성값이 주어지지 않을 때 사용되는 초기값)
 - require : 슬롯에 들어갈 수 있는 값이 만족해야 할 제약조건

CPU	default	Intel
	data-type	string
	require	Intel AMD ARM SPARC
memory	data-type	integer

프레임

❖ 프레임의 구성요소 – Cont.

▪ 데몬(demon) 또는 프로시저(procedure)

- 지정된 조건을 만족할 때 실행할 절차적 지식(procedure)을 기술
- 슬롯 값으로 (데몬 실행조건, 데몬 이름)의 쌍
- 데몬 실행조건의 예
 - if_needed : 슬롯 값을 알아야 할 때 (즉, 사용하려고 할 때)
 - if_added : 슬롯 값이 추가될 때
 - if_removed : 슬롯 값이 제거될 때
 - if_modified : 슬롯 값이 수정될 때

HDD	value	512GB
price	if-needed	look-up-the-list
stock	if-needed	ask-for-vendor

↑
데몬 실행조건

↑
데몬 이름

프로시저 또는 데몬(demon)

- ❖ "if-needed" 프로시저는 빈 슬롯에 어떤 값이 필요해질 때에 실행된다 (read).
- ❖ "if-added" 프로시저는 새로운 정보가 그 슬롯에 추가되어야 할 때 실행된다 (write).
- ❖ "if-deleted" 프로시저는 어떤 값이 슬롯으로부터 제거될 때 실행된다. 이 변경에 따라 다른 슬롯의 값도 변경할 필요가 있을 것이다.

프레임

❖ 프레임의 종류(frame type)

- **클래스(class) 프레임**
 - 부류(class)에 대한 정보 표현
- **인스턴스(instance) 프레임**
 - 특정 객체에 대한 정보 표현

❖ 프레임 계층구조(hierachy)

- **상위 프레임**
 - 클래스를 나타내는 프레임
- **하위 프레임**
 - 하위 클래스 프레임 또는 상위 클래스 프레임의 객체
 - 상위 프레임을 **상속**(inheritance) 받음

프레임과 상속

- ❖ 프레임도 클래스 프레임(class frame) 과 인스턴스 프레임(instance frame) 으로 나눌 수 있다.

클래스	컴퓨터
모델	
CPU	
RAM	[기본값] 8MB
그래픽 카드	
가격	

인스턴스	IBM PC 2019 버전
클래스	컴퓨터
모델	IBM PC 2019
CPU	INTEL i9
RAM	32MB
그래픽 카드	GEFORCE
가격	2000000

프레임

- ❖ 프레임 표현의 예 : 컴퓨터
 - 클래스 프레임 Computer

```
(frame
  (frame-name Computer)
  (frame-type class)
  (CPU (default Intel)
    (data-type string)
    (require (Intel AMD ARM SPARC)))
  (OS (default Windows)
    (data-type string))
  (memory (data-type integer))
  (warranty (default 3years))
  (HDD (default 1TB))
  (price (data-type integer))
  (stock (default in-stock)))
```

frame-name	Computer	
frame-type	Class	
CPU	default	Intel
	data-type	string
	require	Intel AMD ARM SPARC
OS	default	Windows
	data-type	string
memory	data-type	integer
warranty	default	3years
HDD	default	1TB
price	data-type	integer
stock	default	in-stock

프레임

❖ 프레임 표현의 예 : 컴퓨터

▪ 인스턴스 프레임 Ultra-Slim-Notebook

frame-name	Computer	
frame-type	Class	
CPU	default	Intel
	data-type	string
	require	Intel AMD ARM SPARC
OS	default	Windows
	data-type	string
memory	data-type	integer
warranty	default	3years
HDD	default	1TB
price	data-type	integer
stock	default	in-stock

(frame

(frame-name Ultra-Slim-Notebook)

(frame-type **instance** (class Computer))

(CPU (value ARM))

(OS (value Android))

(memory (value 4G))

(HDD (value 512GB))

(price (if-needed look-up-the-list)

(stock (if-needed ask-for-vendor)))

(**warranty** 3years) ?

frame-name	Ultra-Slim-Notebook	
frame-type	instance (class Computer)	
CPU	value	ARM
OS	value	Android
memory	value	4G
warranty	value	3years
HDD	value	512GB
price	if-needed	look-up-the-list
stock	if-needed	ask-for-vendor

← 데몬

프레임

❖ 프레임과 규칙을 결합한 지식 표현

- 프레임은 특정 개념이나 대상에 대한 속성들 표현
 - 관련된 속성들을 하나의 덩어리로 관리
- 규칙을 사용하여 조건적인 지식 표현
 - 데몬에 규칙 사용
 - 또는 규칙의 조건부나 결론부에서 프레임 사용
- 대부분의 규칙기반 시스템에서 객체(object) 개념 사용
 - 객체의 표현에 프레임 사용 가능

프레임

❖ 프레임 vs 클래스와 객체

▪ 클래스와 객체

- 소프트웨어 개발에 있어서 **모듈화**, **재사용성** 및 **유지보수**의 용이성을 고려한 프로그래밍 개념
- **정보은닉** 등 **정보 접근**에 대한 제한 메커니즘

▪ 프레임

- 사람이 특정 **대상**에 대해 갖는 **지식의 표현**을 목표
- 슬롯의 특정 상황에 따라 **자동**으로 **호출**되는 **데몬** 개념

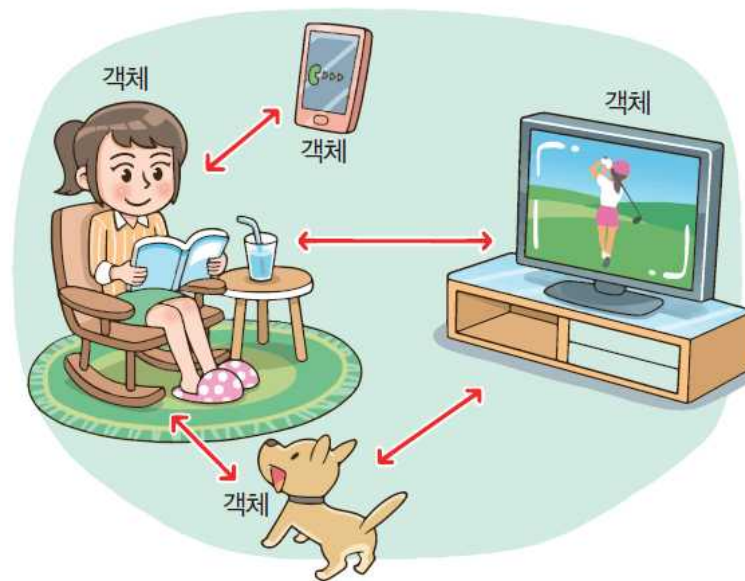
프레임의 장점

1. 개념을 구조화하여 표현할 수 있다. 프레임은 개념을 구조화하여 표현하기 때문에, 개념과 개념 간의 관계, 속성, 기능 등을 명확하게 표현할 수 있다. 이로 인해 개념의 이해와 유추가 쉬워지며, 지식의 구성과 구조를 명확하게 파악할 수 있다.
2. 지식의 재사용성이 높다. 프레임은 유사한 개념을 쉽게 만들 수 있기 때문에, 지식의 재사용성이 높다. 새로운 개념을 만들 때, 이미 만들어진 프레임을 조합하여 새로운 프레임을 만들 수 있으며, 이를 통해 지식을 쉽게 확장할 수 있다.
3. 지식의 일관성을 유지할 수 있다. 프레임은 개념을 구조화하여 표현하기 때문에, 개념 간의 관계와 속성이 일관성을 유지한다. 이로 인해 지식의 일관성을 유지할 수 있으며, 오류와 모순을 방지할 수 있다.

프레임과 객체 지향 프로그래밍

26

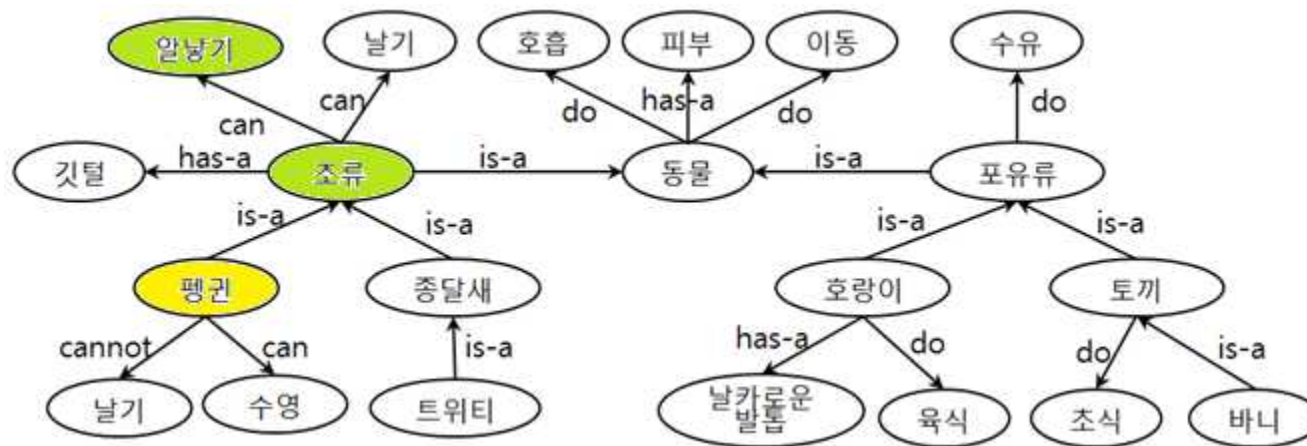
- 인공지능 분야에서는 프레임이란 용어로 객체를 나타낸다.
- 객체의 필드에 해당하는 것이 프레임의 슬롯이다.
- 프레임의 슬롯은 값뿐만 아니라 프로시저(demon)도 가질 수 있다. 슬롯에 붙은 프로시저가 바로 객체의 메소드라고 볼 수 있다.



의미망과 프레임

27

- 의미망의 프레임으로의 변환
 - 노드별로 프레임 생성



frame-name	펭귄
frame-type	Class
is-a	조류
can	수영
cannot	날기

frame-name	조류
frame-type	Class
is-a	동물
can	날기 알내기
has-a	깃털

시멘틱 웹

- 기존 웹의 문제점:
 - 사람만 액세스할 수 있다는 점이다.
 - 컴퓨터가 네트워크를 통해 정보를 전송 받고 화면에 표시하지만, 올바른 정보를 선택하는 데 컴퓨터는 큰 도움이 되지 않는다.
 - 예를 들어서 검색엔진이 발달하여서 사용자가 키워드를 입력하면 많은 검색 결과를 보여주지만, 아직은 사용자가 검색 결과를 클릭해서 내용을 읽기 전까지는 문서에 대한 정보를 올바르게 알 수 없다.
- 그 이유는 무엇일까?
 - 컴퓨터가 아직까지 웹 페이지의 구조와 레이아웃만 이해하고, 의미를 파악하지 못하기 때문이다

기존 웹의 문제점

아 스마트폰 X99 상품의
가격이 100만원이라고.



```
<html>
<body>
<div>
<span> Smartphone </span>
<span> X99 </span>
<span> 100만원 </span>
</div>
...
```

“Smartphone” 옆에 “X99”와
“100만원” 문자열이 옆에 있
네. 무슨 의미일까?



시맨틱 웹

- 시맨틱 웹은 컴퓨터가 정보의 의미를 이해하고 추론할 수 있는 지능형 웹이다. 웹 3.0이라고도 하는 시맨틱 웹(semantic web)은 W3C에서 설정한 표준을 통해 웹을 확장한 것이다.
- 시맨틱 웹의 목표는 인터넷 상의 데이터를 컴퓨터들이 읽고 이해할 수 있도록 만드는 것이다. 웹의 창시자 팀 버너스 리(Tim Berners Lee)는 1999년에 시맨틱 웹에 대한 자신의 비전을 다음과 같이 표현했다

나는 웹이 웹상의 데이터를 분석할 수 있게 되는 꿈을 가지고 있다. 이것을 가능하게 하는 "시맨틱 웹"은 아직 등장하지 않았지만, 등장하면 일상적인 거래는 기계와 기계가 서로 소통하여서 자동적으로 처리될 것이다. 사람들이 오랫동안 기다려온 "지능형 에이전트"가 마침내 실현될 것이다.“



Semantic Web

31

- 시맨틱 웹(**Semantic Web**)은 팀 버너스 리(Tim Berners-Lee)가 처음 제안한 개념으로, 기계가 데이터를 이해할 수 있도록 웹 상의 데이터를 구조화하고 의미를 부여하는 것을 목표로 한다. 이를 통해 더 지능적이고 자동화된 상호작용을 가능하게 하려는 것이 목적.
- Current Status:
 1. 기술과 표준의 발전 :
- **RDF (Resource Description Framework), OWL (Web Ontology Language), SPARQL** 등의 기술이 널리 채택되었으며, 웹 상의 구조화된 데이터를 표현하고 쿼리하는 핵심 기술로 자리 잡았다.
- 링크드 데이터(Linked Data)는 시맨틱 웹의 주요 성공 사례 중 하나이다. **DBpedia, Wikidata, schema.org**와 같은 대규모 데이터 세트가 공개되어 있으며, 이를 통해 다양한 분야의 데이터 간 관계를 기계가 이해할 수 있게 되었다.

2. 적용 사례 :

- **검색 엔진:** 구글, 마이크로소프트, 야후 등은 구조화된 데이터를 활용해 검색 결과를 향상시키고 있다. 예를 들어 구글의 지식 그래프(Knowledge Graph)는 구조화된 데이터를 사용해 검색 쿼리를 더 잘 이해하고 있다.
- **의료 분야:** 온톨로지(ontologies)와 데이터 모델을 통해 의료 지식을 통합하여 더 나은 진단과 의사결정 시스템을 지원하고 있다.
- **전자 상거래:** 구조화된 데이터가 제품 검색성을 향상시키고, 아마존과 같은 플랫폼에서 더 관련성 높은 추천을 제공하는 데 기여하고 있다.

3. 과제와 한계 :

- **데이터 가용성:** 큰 규모의 데이터 세트가 존재하지만, 웹 상의 많은 데이터는 여전히 비구조화되어 있으며, 메타데이터가 부족해 시맨틱 웹에 완전히 통합되지 못하고 있다.
- **도입 속도:** 시맨틱 웹 기술의 도입은 복잡한 표준과 비용 문제로 인해 다소 더디게 진행되고 있다.
- **확장성과 추론:** 대규모 복잡한 데이터 세트에서 기계가 추론할 수 있는 능력을 확장하는 것은 여전히 과제로 남아 있다. 특히 일관성 없거나 불완전한 데이터를 처리하는 데 어려움이 있다.

- 결론 :

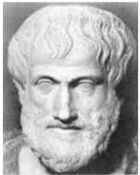
- 시맨틱 웹은 일부 실현된 상태이다. RDF, OWL, Linked data, Microdata, JSON-LD 등을 통해 검색 엔진, 지식 그래프, 의료 분야에서 발전을 이루었지만, 웹 전체가 기계가 완벽하게 이해할 수 있는 상태가 되기까지는 아직 갈 길이 남아 있다. 데이터 가용성 문제와 표준화, 도입 속도 등의 도전 과제가 남아 있으며, 이 비전은 계속 진화하고 있다.
- 시맨틱 웹은 원래 구상했던 형태 그대로는 실현되지 않았지만, 그 핵심 아이디어와 기술들은 현대 웹과 데이터 시스템에 깊이 스며들어 있다. "시맨틱 웹"이라는 용어 자체는 점점 덜 사용되지만, 의미론적 데이터, 지식 그래프, 구조화된 데이터라는 개념들은 오히려 더 중요해지고 있다.

05 논리

❖ 논리(論理, logic)

- 말로 표현된 **문장**들에 대한 타당한 추론을 위해,
기호를 사용하여 문장들을 **표현**하고
기호의 **조작**을 통해 문장들의 **참** 또는 **거짓**을 **판정**하는 분야

❖ 논리학의 역사



- **아리스토텔레스**(Aristotle, BC384-BC322)
 - 기호의 대수적 조작을 통해 추론을 하는 **삼단 논법**(syllogism) 도입



- **부울**(George Boole, 1815-1864)
 - **명제 논리**(propositional logic)의 이론적 기초 확립

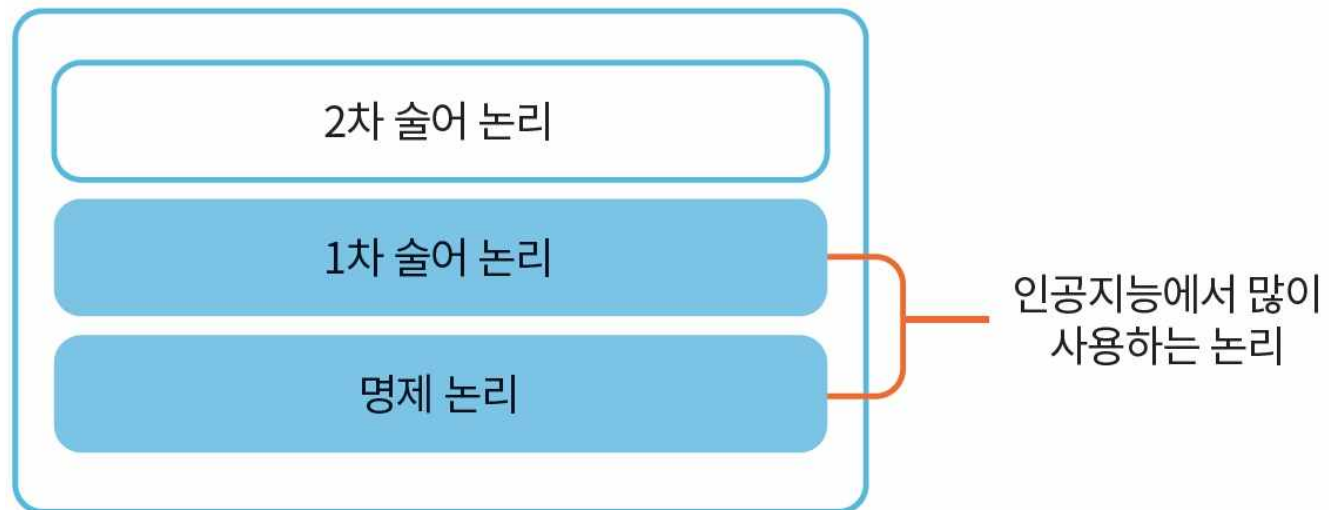


- **프리게**(Gottlob Frege, 1848-1925)
 - **술어 논리**(predicate logic)의 이론적 기초를 확립

05 논리

35

전통적인 논리



장점과 단점

36

- 수학적 근거를 바탕으로 논리 개념을 자연스럽게 표현할 수 있다.
 - 지식의 정형화 영역에 적합하다. 예를 들어서 정리 증명(theorem proving) 기법을 사용할 수 있다.
 - 지식의 첨가와 삭제가 용이하고 비교적 단순하다.
-
- 절차적인 지식 표현이 어렵다.
 - 사실의 구성 법칙이 부족하므로 실세계의 복잡한 구조를 표현하기 어렵다.

06 명제 논리

37

- 기호 논리학에서 명제(proposition)는 참(true, T)이거나 또는 거짓(false, F)을 판별할 수 있는 문장이다.

P = 마트는 월요일부터 토요일까지 영업한다.

Q = 오늘은 일요일이다.

R = 오늘 마트는 영업하지 않는다.

- 논리 연산자 사용 가능: 단순 명제들을 결합해 복합 명제를 만듦

Q = 오늘은 일요일이다.

$NOT\ Q$ = 오늘은 일요일이 아니다.

J = 옷은 파랑색이다.

K = 옷은 스트라이프 무늬가 있다.

$L = J\ AND\ K$ = 옷은 파랑색이고 스트라이프 무늬가 있다.

함축 (함의)

38

$C = \text{오늘은 휴일이다.}$
 $D = \text{오늘은 수업이 없다.}$
 $E = C \rightarrow D$

표 5-4 복합 명제의 논리표

A	B	NOT A	A AND B	A OR B	$A \rightarrow B$
T	T	F	T	T	T
T	F	F	F	T	F
F	T	T	F	T	T
F	F	T	F	F	T

명제논리에서의 추론

39

- 추론(**inference**)이란 우리가 가지고 있는 지식(규칙)과 우리가 이미 알고 있는 사실로부터 새로운 사실을 유추해 내는 것이다.

(지식) 우리집 강아지는 집안에 있거나 앞마당에 있다.

(사실) 강아지가 집안에 없다.

=====

(추론된 사실) 따라서 강아지는 앞마당에 있을 것이다.

추론법칙

40

- 모더스 포넨스(Modus Ponens)

규칙 / $A \rightarrow B$

사실 / A

결론 B

"홍길동이 세계 일주 중이라면 $\rightarrow$ 로또에 당첨된 것이다. "

"홍길동은 세계 일주 중이다."

$\therefore$ "홍길동은 로또에 당첨된 것이다. "

추론법칙

41

- 부정 논법 (Modus Tollens)

규칙 / $A \rightarrow B$

사실 / $NOT B$

결론 $NOT A$

"어떤 동물이 강아지라면 $\rightarrow$ 어떤 동물은 4개의 다리를 가지고 있다."

"어떤 동물은 4개의 다리를 가지고 있지 않다."

$\therefore$ "어떤 동물은 강아지가 아니다."

추론법칙

42

- 삼단논법(syllogism)

규칙 / $A \rightarrow B$

사실 / $B \rightarrow C$

결론 $A \rightarrow C$

“소크라테스는 인간이다”

“인간은 모두 죽는다”

$\therefore$ “소크라테스는 죽는다”

명제 논리

❖ 명제 논리 (propositional logic)

- 명제(命題, proposition)
 - 참, 거짓을 분명하게 판정할 수 있는 문장
 - 아리스토텔레스는 플라톤의 제자이다. (명제)
 - $1+1 = 3$. (명제)
 - 일어나서 아침 먹자. (명제 아님)
- 명제를 **P, Q**등과 같은 **기호로 표현**
- 명제 기호의 **진리값**(truth value)을 사용하여 명제들에 의해 표현되는 문장들의 진리값 결정
- 문장 자체의 **내용**에 대해서는 **무관심**, 문장의 **진리값**에만 **관심**

명제 논리

- ❖ **기본 명제**(primitive proposition)
 - 하나의 진술(statement)로 이루어진 최소 단위의 명제
- ❖ **복합 명제**(compound proposition)
 - 기본 명제들이 결합되어 만들어진 명제
- ❖ 예.
 - 알렉산더는 아시아를 넘본다 $\Rightarrow P$
 - 징기스칸은 유럽을 넘본다 $\Rightarrow Q$
 - 알렉산더는 아시아를 넘보고, 징기스칸은 유럽을 넘본다 $\Rightarrow P \wedge Q$

명제 논리의 구문

❖ 논리식(logical expression)

- 명제를 기호로 표현한 형식
- **명제기호**, 참과 거짓을 나타내는 **T**와 **F**, 명제 기호를 연결하는 **논리기호**인 $\neg$, $\vee$, $\wedge$, $\rightarrow$, $\equiv$ 를 사용하여 구성

표 3.1 논리기호

논리기호	이름	논리식	의미
$\neg$	부정(negation)	$\neg P$	P 가 아님
$\vee$	논리합(disjunction)	$P \vee Q$	P 또는 Q
$\wedge$	논리곱(conjunction)	$P \wedge Q$	P 그리고 Q
$\rightarrow$	함의(implication)	$P \rightarrow Q$	P 이면 Q
$\equiv$	동치(equivalence)	$P \equiv Q$	$(P \rightarrow Q) \wedge (Q \rightarrow P)$

명제 논리의 구문

❖ 리터럴(literal)

- 명제 기호 P 와 명제 기호의 부정 $\neg P$

❖ 절(clause)

- 리터럴들이 **논리합**으로만 연결되거나 **논리곱**으로 연결된 논리식

$$P \vee Q \vee \neg R \quad (\text{논리합 절})$$

$$P \wedge Q \wedge \neg R \quad (\text{논리곱 절})$$

❖ 논리곱 정규형 (conjunctive normal form, CNF)

- 논리합 절들이 논리곱으로 연결되어 있는 논리식

$$(P \vee Q \vee \neg R) \wedge (\neg Q \vee R \vee S) \wedge (P \vee R \vee S)$$

❖ 논리합 정규형 (disjunctive normal form, DNF)

- 논리곱 절들이 논리합으로 연결되어 있는 논리식

$$(P \wedge Q \wedge \neg R) \vee (\neg Q \wedge R \wedge S) \vee (P \wedge R \wedge S)$$

명제 논리의 구문

❖ 정형식(well-formed formula, wff)

- 논리에서 문법에 맞는 논리식
- 명제 논리에 대한 정형식

(1) 진리값 T, F와 명제 기호들 $P, Q, R, \dots$ 은 정형식이다.

(2) p 와 q 가 정형식이면, 논리 기호를 사용하여 구성되는 논리식 $\neg p$, $p \vee q$,
 $p \wedge q$, $p \rightarrow q$, $p \equiv q$ 도 정형식이다.

(3) (1)과 (2)에 의해 정의되는 논리식만 정형식이다.

$$(P \wedge Q) \rightarrow \neg P$$

$$P \rightarrow \neg P$$

$$P \vee P \rightarrow P$$

$$(P \rightarrow Q) \rightarrow (\neg Q \rightarrow \neg P)$$

명제 논리의 의미

❖ 진리표(truth table)

- 논리기호에 따라 참, 거짓 값을 결합하는 방법을 나타낸 표

표 3.2 진리표

P	Q	$\neg P$	$P \vee Q$	$P \wedge Q$	$P \rightarrow Q$	$P \equiv Q$
F	F	T	F	F	T	T
F	T	T	T	F	T	F
T	F	F	T	F	F	F
T	T	F	T	T	T	T

❖ 논리식의 해석(interpretation)

- 논리식의 진리값을 결정하는 것
 - $P = T, Q = F, R = T$
 $(P \vee \neg Q) \wedge (Q \vee \neg R) = F$
- 우선 각 명제기호의 진리값 결정 필요
 - 명제 기호에 "명제"를 대응시키고, 해당 명제의 진리값을 결정
 예. $P \Rightarrow$ "토마토는 과일이다", $P = F$
 - 대응된 명제를 명제 기호의 외연(外延) 또는 의미(denotation)라 함

명제 논리의 의미

❖ 논리식의 **모델**(model)

- 논리식의 **명제기호**에 참값(T) 또는 거짓값(F)을 할당한 것
 - $(P \vee \neg Q) \wedge (Q \rightarrow \neg R) : P = T, Q = F, R = T$
- 모델이 주어지면, **진리표**를 사용하여 논리식의 진리값 결정, 즉 **해석** 가능

P	Q	$\neg P$	$P \vee Q$	$P \wedge Q$	$P \rightarrow Q$	$P \equiv Q$
F	F	T	F	F	T	T
F	T	T	T	F	T	F
T	F	F	T	F	F	F
T	T	F	T	T	T	T

- n 개의 명제기호가 논리식에 사용된다면, 각각 T 또는 F값을 가질 수 있기 때문에, 총 2^n 개의 모델이 존재

❖ A, B, C가 명제 기호일 때, 다음 논리식에 대한 모델의 개수를 구하시오.

1. $A \vee (B \wedge \neg C)$

2. $A \wedge \neg C$

명제 논리의 의미

❖ 타당한 논리식 (valid logical expression)

- 모든 가능한 모델에 대해서 항상 참(T)인 논리식
- 항진식(恒眞式, tautology)

- 예. $P \vee \neg P$

$$P = \text{T인 경우} : P \vee \neg P = \text{T}$$

$$P = \text{F인 경우} : P \vee \neg P = \text{T}$$

❖ 항위식(恒僞式, contradiction)

- 모든 가능한 모델에 대해서 항상 거짓이 되는 논리식

- 예. $P \wedge \neg P$

$$P = \text{T인 경우} : P \wedge \neg P = \text{F}$$

$$P = \text{F인 경우} : P \wedge \neg P = \text{F}$$

명제 논리의 의미

❖ 충족가능한(satisfiable) 논리식

- 참으로 만들 수 있는 모델이 하나라도 있는 논리식
 - 예. $(P \vee \neg Q) \wedge (Q \vee \neg R)$
 $P = \text{T}, Q = \text{T}, R = \text{F}$

❖ 충족불가능한(unsatisfiable) 논리식

- 참으로 만들 수 있는 모델이 전혀 없는 논리식
- 항위식인 논리식
 - 예. $P \wedge \neg P$

❖ 다음 논리식이 타당한지 (valid), 충족불가능한지 (unsatisfiable) 답하시오.

1. $S \rightarrow S$

2. $(S \rightarrow U) \rightarrow (\neg S \rightarrow \neg U)$

3. $S \vee U \vee \neg U$

4. $S \vee U \vee (S \rightarrow U)$

$$\begin{aligned} & \text{not(not } S \text{ or } U) \text{ or } (S \text{ or not } U) \\ &= (S \text{ and not } U) \text{ or } (S \text{ or not } U) \\ &= (S \text{ or not } U) \text{ and } (S \text{ or not } U) \\ &= S \text{ or not } U \end{aligned}$$

명제 논리의 의미

❖ 동치관계(equivalence relation)의 논리식

- 어떠한 모델에 대해서도 같은 값을 갖는 두 논리식

$$(1) \neg(\neg p) \equiv p$$

$$(2) p \vee F \equiv p, \quad p \wedge T \equiv p$$

$$(3) p \vee \neg p \equiv T, \quad p \wedge \neg p \equiv F$$

$$(4) \neg(p \wedge q) \equiv \neg p \vee \neg q, \quad \neg(p \vee q) \equiv \neg p \wedge \neg q$$

$$(5) p \rightarrow q \equiv \neg p \vee q$$

$$(6) p \vee (q \vee r) \equiv (p \vee q) \vee r, \quad p \wedge (q \wedge r) \equiv (p \wedge q) \wedge r$$

$$(7) p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r), \quad p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$$

임의의 정형식을 절의 논리곱으로 변환하기

❖ 명제논리의 임의의 정형식은 이와 동치인 절의 논리곱으로 변환 가능

❖ 정규형

- 논리곱 정규형(conjunctive normal form, CNF): 절(논리합)의 논리곱으로 나타낸 정형식
- 논리합 정규형(disjunctive normal form, DNF): 논리곱 절의 논리합으로 나타낸 정형식

임의의 정형식을 CNF로 변환하는 절차

예제) $\neg(P \supset Q) \vee (R \supset P)$

1. 함의 기호($\supset$)를 $\vee$ 를 이용한 동치식으로 변환: $\neg(\neg P \vee Q) \vee (\neg R \vee P)$
2. 이중 $\neg$ 기호 삭제 (드모르간 법칙): $(P \wedge \neg Q) \vee (\neg R \vee P)$
3. 결합법칙과 배분법칙을 이용하여 CNF로 변환:

$$(P \vee \neg R \vee P) \wedge (\neg Q \vee \neg R \vee P)$$

$$(P \vee \neg R) \wedge (\neg Q \vee \neg R \vee P)$$

- 절의 논리곱(CNF 형태)은 절의 집합으로 표시: $\{(P \vee \neg R), (\neg Q \vee \neg R \vee P)\}$

명제 논리의 의미

❖ 동치관계를 이용한 논리식의 변환

- 논리식의 동치관계를 이용하면

임의의 논리식을 논리곱 정규형(CNF)과 같은 정형식으로 변환

$$\begin{aligned}(p \wedge (q \rightarrow r)) &\rightarrow p \wedge q \\ &\equiv \neg(p \wedge (\neg q \vee r)) \vee (p \wedge q) \\ &\equiv (\neg p \vee \neg(\neg q \vee r)) \vee (p \wedge q) \\ &\equiv (\neg p \vee (q \wedge \neg r)) \vee (p \wedge q) \\ &\equiv ((\neg p \vee q) \wedge (\neg p \vee \neg r)) \vee (p \wedge q) \\ &\equiv ((\neg p \vee q) \vee (p \wedge q)) \wedge ((\neg p \vee \neg r) \vee (p \wedge q)) \\ &\equiv ((\neg p \vee q) \vee p) \wedge ((\neg p \vee q) \vee q) \wedge ((\neg p \vee \neg r) \vee p) \wedge ((\neg p \vee \neg r) \vee q) \\ &\equiv (T \vee q) \wedge (\neg p \vee q) \wedge (T \vee \neg r) \wedge (\neg p \vee \neg r \vee q) \\ &\equiv (\neg p \vee q) \wedge (\neg p \vee q \vee \neg r)\end{aligned}$$

귀결 (entailment)

❖ 집합 Δ 에 있는 모든 wff를 True로 하는 해석에 대해 wff ω 가 True값을 가질 때

- $\Delta \models \omega$ 로 표시
- Δ 는 ω 를 논리적으로 귀결한다(logically entails)
- ω 는 Δ 를 논리적으로 따른다(logically follows)
- ω 는 Δ 의 논리적 결론(logical consequence)
예) $\{P\} \models P$, $\{P, P \supset Q\} \models Q$, $F \models \omega$ (ω 는 임의의 wff), $P \wedge Q \models P$

명제 논리의 의미

❖ 논리적 귀결(logical entailment)

- Δ : 정형식(wff)의 집합. $\Delta = \{P, P \rightarrow Q\}$
 ω : 정형식. $\omega = Q$
- Δ 에 있는 모든 정형식을 참(T)으로 만드는 모델이, ω 를 참(T)으로 만든다
 $= \Delta$ 는 ω 를 논리적으로 귀결한다(logically entail)
 $= \omega$ 는 Δ 를 논리적으로 따른다(logically follow)
 $= \omega$ 는 Δ 의 논리적 결론(logical consequence)이다

▪ 표기법 : $\Delta \models \omega$

- $\{P, P \rightarrow Q\} \models Q$

P	Q	$\neg P$	$P \vee Q$	$P \wedge Q$	$P \rightarrow Q$	$P \equiv Q$
F	F	T	F	F	T	T
F	T	T	T	F	T	F
T	F	F	T	F	F	F
T	T	F	T	T	T	T

- Δ 가 참이면, ω 도 참이다
- **추론**
 - 참으로 알려진 Δ 로부터, 알려지지 않은 참인 ω 를 찾는 것

명제 논리의 추론

❖ 추론(推論, inference)

▪ 귀납적 추론(inductive inference)

- 관측된 복수의 사실들을 일반화(generalization)하여 일반적인 패턴 또는 명제를 도출하는 것

▪ 연역적 추론(deductive inference)

- 참인 사실들 또는 명제들로부터 새로운 참인 사실 또는 명제를 도출하는 것

▪ 논리에서의 추론

- 연역적 추론의 형식화된 버전
- 함의($\rightarrow$)의 논리적 관계를 이용하여 새로운 논리식을 유도해 내는 것
- 함의 $p \rightarrow q$
 - p : 전제(premise)
 - q : 결론(conclusion, consequence)
 - p 가 참일 때, q 가 참임

명제 논리의 추론

❖ 추론규칙(inference rule)

- 참인 논리식들이 논리적으로 귀결하는 새로운 논리식을 만들어내는 기계적으로 적용되는 규칙

- 긍정 논법 (modus ponens)

추론 표기법: $\vdash$

$p \rightarrow q$	새이다 $\rightarrow$ 날 수 있다
p	새이다
-----	-----
q	날 수 있다

$$p \rightarrow q, p \vdash q$$

- 부정 논법 (modus tollens)

$p \rightarrow q$	새이다 $\rightarrow$ 날 수 있다
$\neg q$	날 수 없다
-----	-----
$\neg p$	새가 아니다

$$p \rightarrow q, \neg q \vdash \neg p$$

- 삼단 논법 (syllogism)

$p \rightarrow q$	새이다 $\rightarrow$ 날개가 있다
$q \rightarrow r$	날개가 있다 $\rightarrow$ 날 수 있다
-----	-----
$p \rightarrow r$	새이다 $\rightarrow$ 날 수 있다

$$p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$$

명제 논리의 추론

❖ 추론규칙 – cont.

▪ 논리 융합 (resolution)

• 일반화된 추론규칙

– 긍정 논법, 부정 논법, 삼단 논법의 규칙을 포함한 추론 규칙

- 두 개의 논리합절이 같은 기호의 긍정과 부정의 리터럴을 서로 포함하고 있을 때, 해당 리터럴들을 제외한 나머지 리터럴들의 논리합절을 만들어 내는 것

$$P \vee q, \neg P \vee r \vdash q \vee r$$

↑

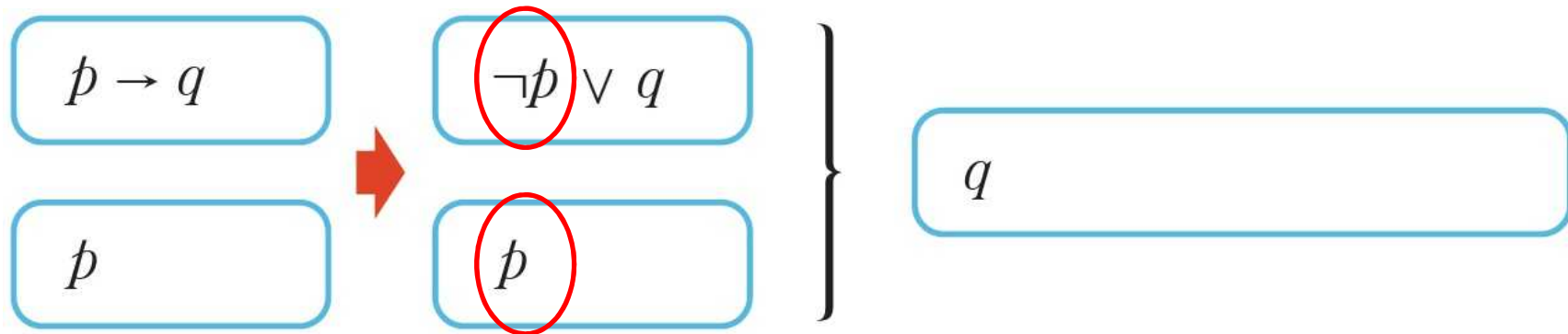
논리융합식(resolvent)

논리융합

- ❖ 논리융합은 같은 기호의 리터럴과 부정 리터럴을 가지고 있는 2개의 절을 조합하여서 새로운 절을 생성하는 방법이다.



논리융합으로 모더스 포넌스를 유도해보자.



정당성과 완전성

❖ 추론과 귀결의 연관

- $\mathfrak{R}$: 추론 규칙의 집합, Δ : wff 집합, ω : wff
 1. 임의의 wff 집합 Δ 와 wff ω 에 대해, $\Delta \vdash_{\mathfrak{R}} \omega$ 가 $\Delta \models \omega$ 를 함의하면(implies), 그 추론 규칙의 집합 $\mathfrak{R}$ 을 **정당하다**(sound)고 한다.
 $\mathfrak{R}$ 이 정당하다 : $\Delta \vdash_{\mathfrak{R}} \omega \supset \Delta \models \omega$
 2. 임의의 wff 집합 Δ 와 wff ω 에 대해 $\Delta \models \omega$ 일 때, 추론 규칙의 집합 $\mathfrak{R}$ 을 이용하여 Δ 로부터 ω 의 증명이 존재하면, $\mathfrak{R}$ 은 **완전하다**(complete)고 한다.
 $\mathfrak{R}$ 이 완전하다 : $\Delta \models \omega \supset \Delta \vdash_{\mathfrak{R}} \omega$
- 추론 규칙이 정당하고 완전하면 진리표를 이용하지 않고도 한 wff가 wff 집합을 논리적으로 따르는지 아닌지 결정가능

명제 논리의 추론

❖ 추론 규칙의 정당성과 완전성

▪ 추론 규칙의 **정당성** (sound)

- 추론 규칙에 의해 생성된 논리식은 주어진 논리식들이 논리적으로 귀결하는 것이다.
- 즉, 추론 규칙이 만들어 낸 것은 항상 참이다.

$$\Delta \vdash \omega \rightarrow \Delta \models \omega$$

▪ 추론 규칙의 **완전성** (complete)

- 주어진 논리식들이 논리적으로 귀결하는 것들은 추론 규칙이 찾아 낼 수 있다.

명제 논리의 추론

❖ 정리증명(theorem proving)

- **공리(axiom)**
 - 추론을 할 때, 참인 것으로 주어지는 논리식
- **정리(theorem)**
 - 공리들에 추론 규칙을 적용하여 얻어지는 논리식
- **정리 증명**
 - 공리들을 사용하여 **정리가 참인 것을 보이는 것**
 - **구성적 증명(constructive proof)**
 - 공리들에 추론 규칙들을 적용하여 증명을 만들어 보이는 증명
 - **논리융합 반박(resolution refutation)**
 - 증명할 정리를 부정(negation)한 다음, 논리융합 방법을 적용하여 모순이 발생하는 것을 보여서, 정리가 참임을 증명하는 방법

절에서의 논리융합

- ❖ Σ_1, Σ_2 : 리터럴의 집합, λ : 아톰
 - $\{\lambda\} \cup \Sigma_1$ 과 $\{\neg\lambda\} \cup \Sigma_2$ 로부터 $\Sigma_1 \cup \Sigma_2$ 를 추론
 - $\Sigma_1 \cup \Sigma_2$ 를 두 절의 **논리융합식**(resolvent)이라 부름

예)

- $R \vee P$ 와 $\neg P \vee Q$ 는 $R \vee Q$ 로 논리융합됨: 연결(chaining)
- R 과 $\neg R \vee P$ 는 P 로 논리융합됨: **모더스 포넨스**
- $P \vee Q \vee R \vee S$ 와 $\neg P \vee Q \vee W$ 는 $Q \vee R \vee S \vee W$ 로 논리융합됨
- $P \vee Q \vee \neg R$ 과 $P \vee W \vee \neg Q \vee R$ 의 경우
 - Q 에 대해 논리융합: $P \vee \neg R \vee R \vee W \rightarrow T$
 - R 에 대해 논리융합: $P \vee Q \vee \neg Q \vee W \rightarrow T$
- λ 와 $\neg\lambda$ 를 논리융합하면 공절(empty clause): F 를 추론

논리융합의 정당성(soundness)

- ❖ $\{\lambda\} \cup \Sigma_1$ 과 $\{\neg\lambda\} \cup \Sigma_2$ 가 둘 다 True값을 가지면 논리융합식 $\Sigma_1 \cup \Sigma_2$ 도 True

(증명) “사례별로 추론”

- (사례 1) λ 가 True $\rightarrow \{\neg\lambda\} \cup \Sigma_2$ 절이 True이기 위해서는 Σ_2 가 True
- (사례 2) λ 가 False $\Rightarrow \{\lambda\} \cup \Sigma_1$ 절이 True이기 위해서는 Σ_1 이 True
- 따라서 $\Sigma_1 \cup \Sigma_2$ 는 True

논리융합 반박 (Resolution refutation)

❖ 논리융합 반박의 필요성

- 논리융합은 정당하지만(sound), 완전하지는 않다(not complete)
예) $P \wedge R \models P \vee R$, 하지만 $\{P, R\}$ 로부터 논리융합 방법으로 $P \vee R$ 추론 불가능
- $P \vee R$ 의 부정이 집합 $\{P, R\}$ 과 일치하지 않는다는 것은 논리융합으로 증명 가능
- 논리융합의 반박 완전성을 보여줌으로써 논리융합으로 추론한 결과도 논리적으로 귀결함(참임)을 증명한다

논리융합 반박

- ❖ 모순을 이용한 논리융합 (목표의 부정 $\rightarrow$ 모순 유도 $\rightarrow$ 목표 증명)
예) $P \vee R$ 의 부정: $\{\neg P, \neg R\} \rightarrow \{P, R\}$ 과 논리융합하면 공절이 만들어짐
- ❖ wff의 집합 Δ 로부터 임의의 wff ω 를 증명하기 위한 논리융합 반박 절차
 1. Δ 에 있는 wff들을 절 형태(clause form)로 변환(CNF)
 2. 증명하고자 하는 ω 의 **부정**을 절 형태로 변환
 3. 1과 2단계의 결과인 절을 하나의 집합 Γ 으로 합침
 4. Γ 에 있는 절에 논리융합을 반복 적용: 공절이 나올 때까지 (또는 더 이상 Γ 에 새로운 절이 추가되지 않을 때까지)

명제 논리의 추론

❖ 논리융합 반박을 이용한 정리증명의 예

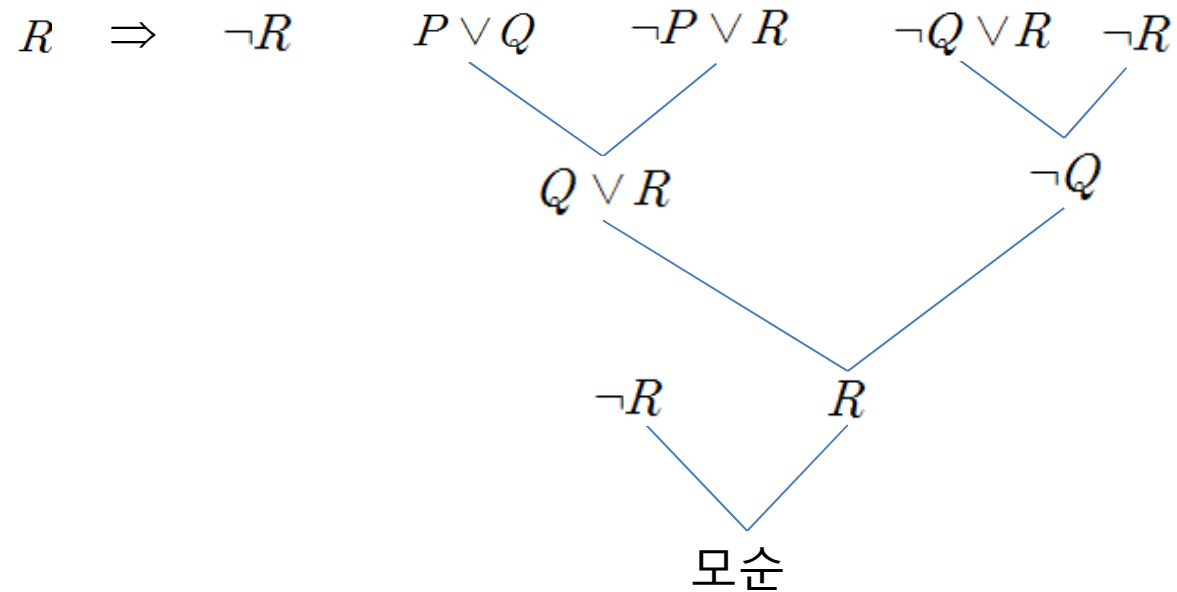
▪ 공리 (axiom)

$$P \vee Q$$

$$P \rightarrow R \quad \neg P \vee R$$

$$Q \rightarrow R \quad \neg Q \vee R$$

▪ 정리 (theorem)



예시 1: 지식 기반의 모순 찾기 (불일치 탐지)

- 문제: 한 조직의 규정이나 스마트홈의 설정이 서로 모순되는지 확인하는 것.

- 지식 기반:

- 1.문이 잠겨 있으면 $\rightarrow$ 알람이 켜진다. ($D \rightarrow A$)
- 2.창문이 열려 있으면 $\rightarrow$ 알람이 꺼진다. ($W \rightarrow \neg A$)
- 3.지금 문이 잠겨 있다. (D)
- 4.지금 창문이 열려 있다. (W)

- 논리융합반박이 보여주는 과정:

- 1.논증 1: (1)과 (3)으로부터 알람이 켜진다(A)를 주장.
- 2.논증 2: (2)와 (4)로부터 알람이 꺼진다($\neg A$)를 주장.
- 3.반박 발생: A 와 $\neg A$ 는 서로 직접적으로 충돌합니다.
- 4.보여줄 수 있는 결론: "지식 기반에 모순이 존재함을 발견. 규칙 (1)과 (2)는 동시에 만족될 수 없는 조건." 이는 단순히 '모순이다'라고 말하는 것이 아니라, 어떤 명제들이 어떻게 충돌하는지 그 구체적인 경로를 보여줌

예시 2: 최선의 결정 또는 가장 방어 가능한 결론 도출

•문제: 상반된 증거가 있을 때, 어떤 결론을 믿어야 하는지 결정.

•지식 기반 (의학적 추론):

1. 발열과 기침이 있으면 $\rightarrow$ 감기일 수 있다(C). ($F \wedge K \rightarrow C$)
2. 맥박이 정상이면 $\rightarrow$ 감기가 아니다($\neg C$). ($N \rightarrow \neg C$)
3. 환자에게 발열이 있다(F).
4. 환자에게 기침이 있다(K).
5. 환자의 맥박이 정상이다(N).

•논리융합반박이 보여주는 과정:

1. 논증 A (감기 주장): (1), (3), (4)로부터 감기이다(C)를 주장.

2. 논증 B (감기 아님 주장): (2)와 (5)로부터 감기가 아니다($\neg C$)를 주장.

3. 반박 분석:

- 논증 B는 논증 A의 결론(C)을 직접 반박함.
- 그런데 논증 B의 전제 (N)을 반박할 수 있는 다른 증거(예: 맥박이 정상이다($\neg N$))가 지식 기반에 없음.
- 따라서 논증 A는 논증 B에 의해 격과당한 상태임.

4. 보여줄 수 있는 결론: " 현재 주어진 정보 하에서는 ' 감기가 아니다($\neg C$) ' 라는 결론이 유일하게 방어 가능한 상태임. ' 감기이다 ' 라는 주장은 ' 맥박이 정상이다 ' 라는 사실에 의해 반박되었음. " 이는 단순한 답이 아니라, 결론에 이르는 논쟁의 흐름과 그 이유를 보여줌.

예시 3: 법적 추론 또는 규정 해석

- 문제: 특정 사건에 적용 가능한 법조문이 여러 개이고 서로 충돌할 때, 어떤 판결이 가장 합리적인지 보여줌.

- 지식 기반:

1. 공공장소에서 폭력을 행사하면 $\rightarrow$ 처벌받는다(P). ($V \rightarrow P$)

2. 정당방위라면 $\rightarrow$ 처벌받지 않는다($\neg P$). ($S \rightarrow \neg P$)

3. A가 공공장소에서 폭력을 행사했다(V).

4. A의 행동은 정당방위였다(S).

- 논리융합반박이 보여주는 과정:

- 앞의 예와 유사하게 처벌받는다(P)와 처벌받지 않는다($\neg P$)를 주장하는 두 논증이 충돌함.

- 이때, 어느 전제(예: '정당방위였다')를 더 믿을 만한 증거로 뒷받침할 수 있는지에 따라 최종적으로 방어 가능한 결론이 결정됨.

- 보여줄 수 있는 결론: "만약 '정당방위(S)'라는 사실에 대한 증거가 압도적이라면, '처벌받지 않는다($\neg P$)'는 결론이 최종적으로 지지됨. 그 이유는 상대 논증을 반박하는 데 성공했기 때문임."

명제 논리의 지식표현

❖ 명제 논리를 이용한 지식 표현

- 문장으로 표현된 지식으로부터 기본 명제들을 추출
- 각 명제에 대해 명제기호 부여
- 기본 명제들의 논리적 연결 관계를 참고하여 대응되는 명제 기호들을 논리기호로 연결하여 논리식 구성

❖ 명제 논리로 표현된 지식에 대한 추론

- 명제 기호가 나타내는 명제의 의미와는 무관
- 대수적인 기호 연산을 통해서 추론 수행

❖ 앞면이면 내가 이기고, 뒷면이면 네가 진다. 이 문장을 명제논리로 표현하고 내가 이긴다는 것을 논리융합을 이용하여 증명하라. 필요하다면 다른 문장을 만들어 사용해도 좋다.

- | | |
|------------------------|------------------------|
| 1. $H \supset IW$ | 1. $\neg H \vee IW$ |
| 2. $Tail \supset YL$ | 2. $\neg Tail \vee YL$ |
| 3. $H \vee Tail$ | 3. $H \vee Tail$ |
| 4. $YL \supset IW$ | 4. $\neg YL \vee IW$ |
| 5. $\neg IW$ | 5. $\neg IW$ |
| 6. $H \vee YL$: 2와 3 | |
| 7. $YL \vee IW$: 6과 1 | |
| 8. IW : 7과 4 | |
| 9. Nil : 8과 5 | |

❖ 다음 명제논리 wff를 CNF 형태로 바꾼 다음 절의 집합으로 표기하라.

$$\begin{aligned}& \neg [((P \vee \neg Q) \supset R) \supset (P \wedge R)] \\& \equiv \neg [\neg((P \vee \neg Q) \supset R) \vee (P \wedge R)] \\& \equiv \neg [\neg(\neg(P \vee \neg Q) \vee R) \vee (P \wedge R)] \\& \equiv (\neg(P \vee \neg Q) \vee R) \wedge \neg(P \wedge R) \\& \equiv ((\neg P \wedge Q) \vee R) \wedge (\neg P \vee \neg R) \\& \equiv ((\neg P \vee R) \wedge (Q \vee R)) \wedge (\neg P \vee \neg R) \\& \equiv ((\neg P \vee (R \wedge \neg R)) \wedge (Q \vee R)) \\& \equiv (\neg P) \wedge (Q \vee R)\end{aligned}$$

$$\{\neg P, Q \vee R\}$$

07 술어논리

79

- 명제 논리에서 하나의 명제가 나누어질 수 없기 때문에 어려움이 있다. 즉 우리는 전체 명제가 참이냐 거짓이냐 만을 말할 수 있다.
 - 예를 들어서 “신호등이 파랑색이다.”라는 명제가 있다면 이것을 “신호등”과 “파랑색이다”로 나눌 수 있다면 아주 편리할 것이다.
- 술어 논리에서는 하나의 명제가 객체(object, 또는 인수)와 술어(predicate)로 나누어진다.
- 변수와 한정사를 사용할 수 있다.
- 이와 같은 술어 논리의 특징 때문에 명제를 사용하는 것보다, 훨씬 더 구체적으로 지식을 표현할 수 있다.

술어논리

80

- 예를 들어보자. “만약 x 가 새라면, x 는 날개를 가질 것이다”라는 규칙이 있다고 하자.
 - 이 규칙을 술어논리로 표현하면 다음과 같다.

$$(\forall x) \{ \text{is_a}(x, \text{Bird}) \rightarrow \text{has}(x, \text{Wings}) \}$$

술어 논리

❖ 술어 논리(predicate logic)

- 명제의 내용을 다루기 위해 **변수**, **함수** 등을 도입하고 이들의 값에 따라 참, 거짓이 결정되도록 명제 논리를 확장한 논리
- **술어**(述語, predicate)
 - 문장의 '주어+서술어'형태에서 **서술어**에 해당
 - 대상의 **속성**이나 **대상 간의 관계**를 기술하는 기호
 - 참(T) 또는 거짓(F) 값을 갖는 함수
 - 예.
 - **Student**(John)
 - **Friend**(John, Mary)

술어 논리의 예

명제 논리: *Kim has a house*

술어 논리: *HAS(Kim, house)*

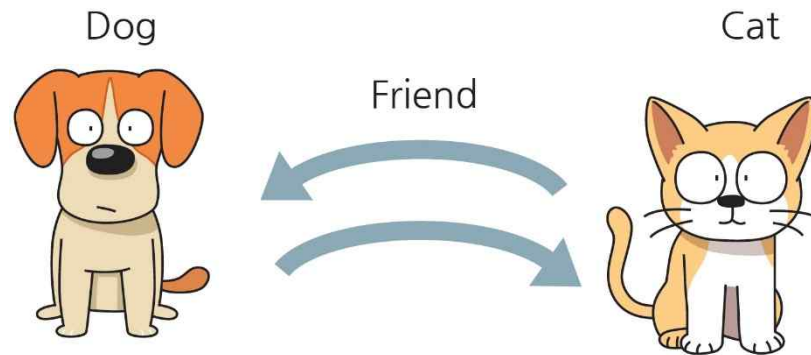
명제 논리: *The orange is yellow*

술어 논리: *YELLOW(orange)*

첫 번째 예에서, 'HAS'는 술어이며, 객체는 'Kim'과 'house'이다. 두 번째 예에서는 'YELLOW'는 술어이고, 'orange'는 객체이다.

객체와 관계

- ❖ 객체는 상수 기호로 나타낸다: 바둑이, 야옹이, Richard, ...
- ❖ 관계는 술어 기호로 나타낸다: HUMAN, DOG, CAT, HAT, ...



변수

- ❖ 예를 들어서 "x가 인간이라면"은 다음과 같이 표현할 수 있다.
 - HUMAN(x)
- ❖ x가 인간이라면 위의 술어 논리식은 참이 된다.

한정사

- ❖ 한정사는 변수의 범위를 서술하는 기호이다.
- ❖ 술어 논리에는 전체 한정사(universal quantifier) $\forall$ 와 존재 한정사(existential quantifier) $\exists$ 를 사용할 수 있다.
 - 전체 한정사 $\forall$ 는 "모든"이라는 의미이다.
 - 존재 한정사 $\exists$ 는 "적어도 하나는 존재"한다는 의미이다.

All dog like cats



$\forall x [\text{DOG}(x) \rightarrow \text{LIKES}(x, \text{cat})]$

Some dogs like cats.



$\exists x [\text{DOG}(x) \rightarrow \text{LIKES}(x, \text{cat})]$

술어 논리의 구문

❖ 술어 논리

- **존재 한정사**(existential quantifier) $\exists$ 와 **전체 한정사**(universal quantifier) $\forall$ 사용
 - 변수의 범위를 고려한 지식을 표현
 - $\exists x \text{ Friend}(\text{John}, x)$
 - 'John은 친구가 한 명은 있다'
 - $\forall x \exists y \text{ Friend}(x, y)$
 - '누구나 친구가 한 명은 있다'

술어 논리의 구문

❖ 함수(function)

- 주어진 인자에 대해서 참, 거짓 값이 아닌 일반적인 값을 반환
- 술어나 다른 함수의 인자로 사용

❖ 항(term)

- 술어의 인자가 될 수 있는 것
- 항이 될 수 있는 것 : 개체상수, 변수, 함수

(1) 개체상수, 변수는 항이다.

(2) $t_1, t_2, \dots, t_n$ 이 모두 항이고, f 가 n 개의 인자를 갖는 함수 기호일 때,

$f(t_1, t_2, \dots, t_n)$ 은 항이다.

(3) (1)과 (2)에 의해 만들어질 수 있는 것만 항이다.

술어 논리의 구문

❖ 술어 논리식에 대한 정형식

- (1) $t_1, t_2, \dots, t_n$ 이 모두 항이고, p 가 n 개의 인자를 갖는 술어 기호일 때, $p(t_1, t_2, \dots, t_n)$ 은 정형식이다.
- (2) p 와 q 가 정형식이면, 논리 기호를 사용하여 구성되는 논리식 $\neg p$, $p \vee q$, $p \wedge q$, $p \rightarrow q$, $p \equiv q$ 도 정형식이다.
- (3) $p(x)$ 가 정형식이고, x 가 변수일 때, $\forall x p(x)$, $\exists x p(x)$ 는 정형식이다.
- (4) (1), (2), (3)에 의해 만들어질 수 있는 것만 술어 논리의 정형식이다.

$$\forall x \forall y \text{Horse}(x) \wedge \text{Dog}(y) \rightarrow \text{Faster}(x, y)$$

$$\exists y \text{Greyhound}(y) \wedge (\forall z \text{Rabbit}(z) \rightarrow \text{Faster}(y, z))$$

$$\text{Horse}(\text{Harry})$$

$$\text{Rabbit}(\text{Ralph})$$

$$\forall y \text{Greyhound}(y) \rightarrow \text{Dog}(y)$$

$$\forall x \forall y \forall z \text{Faster}(x, y) \wedge \text{Faster}(y, z) \rightarrow \text{Faster}(x, z)$$

정형식

- ❖ 우리가 논리융합을 사용하려면 모든 지식이 정형식(WFF: well-formed formula)으로 표현되어야 한다.
- ① 기초 공식(원자)은 정형식이다.
 - 그 외 동등성 술어도 정형식이다.
 - 두 변수나 함수가 같은지를 나타내는 경우
 - $x = y, f(x) = g(x)$
 - **논리 연산자**로 연결된 식이나 변수들도 문법적 조건을 충족하는 경우에는 정형식이 될 수 있지만, 함수나 변수 그 자체만으로는 정형식이 되지 않는다.
- ② P와 Q가 정형식이면 $\neg P, P \vee Q, P \wedge Q, P \rightarrow Q$ 도 정형식이다.
- ③ P가 정형식이면 $\forall x P(x)$ 와 $\exists x P(x)$ 도 정형식이다.
- ④ 정형식은 위의 규칙을 반복하여서 형성가능하다.

정형식

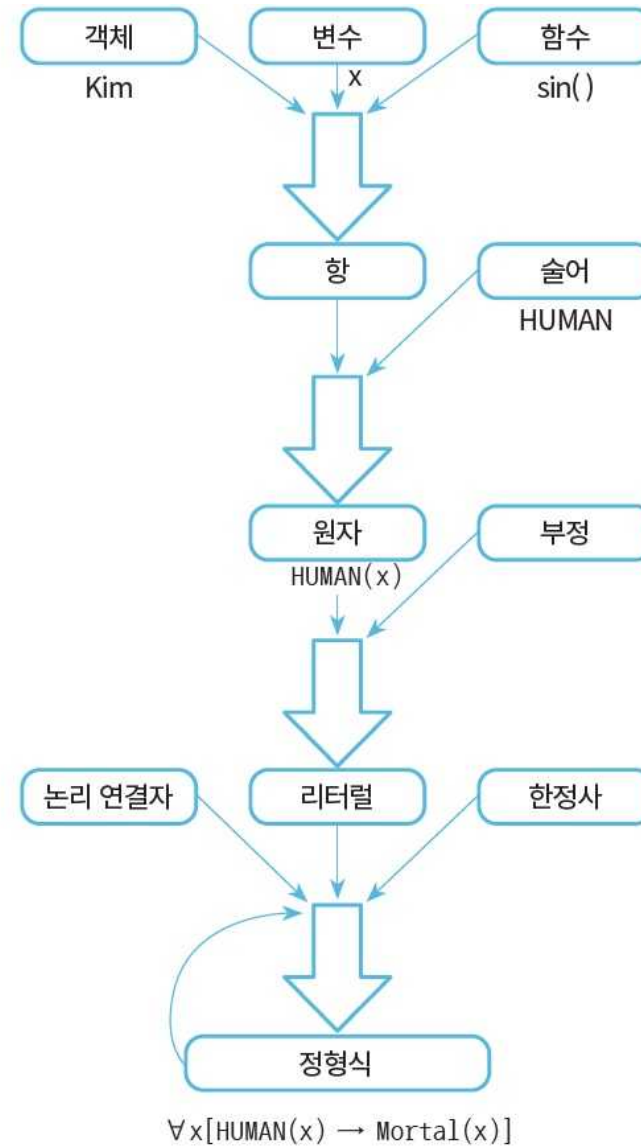


그림 5-6 정형식으로 표현하기

술어 논리의 종류

❖ 일차 술어논리(first-order predicate logic, **FOL**)

- 변수에만 전체 한정사와 존재 한정사를 쓸 수 있도록 한 술어논리

❖ 고차 술어논리(high-order predicate logic)

- 변수뿐만 아니라 함수, 술어기호 등에 대해서 전체 한정사와 존재 한정사를 쓸 수 있도록 한 술어논리
 - $\exists S S(x)$
 - $\exists g \forall x (f(x) = h(g(x)))$: 함수간의 동등 여부로 구성된 술어 정형식
 - 함수들이 어떻게 서로 연결되어 있는지 나타냄
 - 술어 정형식이 반드시 술어만으로 이루어져야 하는 것은 아니며, 함수 관계를 포함하더라도 그 진술이 참 또는 거짓으로 판별 가능하다면 술어 정형식이 될 수 있음
- 고차 술어논리는 수학의 정리 증명, 메타논리적 추론, 그리고 형식적 시스템에서의 추론 등에 사용됨. 일차 술어논리보다 더 풍부하고 강력한 표현력을 가지지만, 결정가능성이 부족할 수 있다.

술어 논리의 지식표현

❖ 술어 논리를 이용한 지식 표현

- (a) Whoever can read is literate. (읽을 수 있으면 문맹이 아니다)
- (b) Monkeys are not literate. (원숭이는 문맹이다)
- (c) Some monkeys are intelligent. (어떤 원숭이는 지능적이다)
- (d) Some who are intelligent cannot read. (지능적이어도 문맹일 수 있다) ← 증명

$$(a) \quad \forall x[CanRead(x) \rightarrow Literate(x)]$$

$$(b) \quad \forall x[Monkey(x) \rightarrow \neg Literate(x)]$$

$$(c) \quad \exists x[Monkey(x) \wedge Intelligent(x)]$$

$$(d) \quad \exists x[Intelligent(x) \wedge \neg CanRead(x)]$$

08 술어논리에서의 추론

93

- 첫 번째 방법은 술어 논리식을 명제 논리식으로 변환한 후에 명제 논리의 추론 기법을 적용하는 것
 - 도메인이 크거나 무한할 경우, 모든 경우를 나열할 수 없어서 실용성이 떨어짐
- 두 번째 방법은 도출, 논리 융합(resolution)이다.

술어 논리의 추론

❖ 단일화(unification) 과정

- 논리융합(resolution)을 적용할 때 대응되는 리터럴이 같아지도록, 변수의 값을 맞춰주는 과정

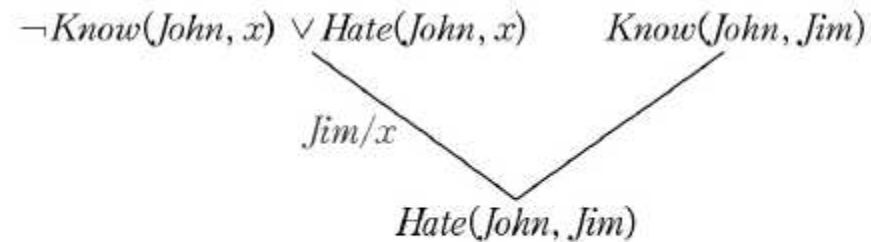


그림 3.5 단일화 과정의 예

x 를 Jim 으로 대체하면 논리융합에 의해 $\text{Hate}(\text{John}, \text{Jim})$ 이 도출됨.

단일화 (unification)

❖ 논리융합

- 두 절이 똑같은 리터럴을 서로 상반되게 포함한 경우

❖ 치환과 단일화의 필요성

- 술어논리에 논리융합을 적용시키기 위해 필요

예) 다음 두 절의 논리융합을 구한다고 가정

1. $P(f(y), A) \vee Q(B, C)$

2. $\neg P(x, A) \vee R(x, C) \vee S(A, B)$

- 두 번째 절에 치환 $\{f(y)/x\}$ 를 적용하면 $\neg P(f(y), A) \vee R(f(y), C) \vee S(A, B)$
- 논리융합을 수행하면 $R(f(y), C) \vee S(A, B) \vee Q(B, C)$ 의 논리융합식 생성

치환(substitution)

- ❖ 정형식의 항: 변수, 객체상수, 함수식(함수상수+항)
- ❖ 정형식의 치환: 변수를 항으로 바꿈
- ❖ 순서쌍의 집합 $s = \{\tau_1/\xi_1, \tau_2/\xi_2, \dots, \tau_n/\xi_n\}$ (τ_i : 항, ξ_i : 변수)
 - τ_i/ξ_i 의 의미: 치환이 미치는 범위 내에 있는 변수 ξ_i 모두를 항 τ_i 로 바꿈
 - 어떤 식 ω 를 s 로 치환한 치환사례(substitution instance)를 ωs 와 같이 표기

예) $s1 = \{z/x, w/y\} \rightarrow P(x, f(y), B) \quad s1 = P(z, f(w), B)$

$s2 = \{A/y\} \rightarrow P(x, f(y), B) \quad s2 = P(x, f(A), B)$

$s3 = \{g(z)/x, A/y\} \rightarrow P(x, f(y), B) \quad s3 = P(g(z), f(A), B)$

$s4 = \{C/x, A/y\} \rightarrow P(x, f(y), B) \quad s4 = P(C, f(A), B)$

❖ 합성

- s_1 과 s_2 에 대한 합성 s_1s_2
 - s_1 의 각 항에 s_2 를 적용하고 여기에 s_1 에서 나타나지 않고 s_2 에서만 나타난 변수를 추가하여 만든 치환
 - $\{g(x,y)/z\} \{A/x, B/y, C/w, D/z\}$
 - $\{g(A,B)/z, A/x, B/y, C/w\}$
- 교환 법칙이 성립하지 않음

❖ $\{\omega\}_s$

- $\{\omega\}_s$ 의 모든 원소에 대하여 치환 s 를 적용하는 경우의 표기

단일화 (unification)

❖ 단일화가 가능하다(unifiable)

- $\omega_1 s = \omega_2 s = \omega_3 s = \dots$ 의 관계가 성립하는 치환 s 가 존재할 때 - 치환 s 를 $\{\omega_i\}$ 의 **단일자(unifier)**라고 함

예) $\{P[x, f(y), B], P[x, f(B), B]\}$ 에 대한 단일자

$$s1 = \{A/x, B/y\}, \quad s2 = \{B/y\}$$

- **가장 일반적인 단일자**(most general unifier, *mg*u) ***g***

- 치환 s 에 의해 정형식의 집합 $\{\omega_i\}$ 가 $\{\omega_i\}s$ 로 단일화된다면 $\{\omega_i\}s = \{\omega_i\}gs'$ 의 관계가 성립하는 치환 s' 이 존재한다.
- *mg*u에 의한 결과는 알파벳 변이를 제외하고는 유일
- UNIFY 알고리즘: *mg*u를 구하는 알고리즘 (불일치 집합의 개념)

```

1: procedure Unify(t1, t2)
2:   Inputs
3:     t1, t2: atoms or terms
4:   Output
5:     most general unifier of t1 and t2 if it exists or 0 otherwise
6:   Local
7:     E: a set of equality statements
8:     S: substitution
9:     E ← {t1=t2}
10:    S = {}
11:    while E ≠ {} do
12:      select and remove  $\alpha = \beta$  from E
13:      if  $\beta$  is not identical to  $\alpha$  then
14:        if  $\alpha$  is a variable and  $\alpha$  is not in  $\beta$  then
15:          replace  $\alpha$  with  $\beta$  everywhere in E and S
16:          S ← { $\beta/\alpha$ } ∪ S
17:        else if  $\beta$  is a variable and  $\beta$  is not in  $\alpha$  then
18:          replace  $\beta$  with  $\alpha$  everywhere in E and S
19:          S ← { $\alpha/\beta$ } ∪ S
20:        else if  $\alpha$  is  $p(\alpha_1, \dots, \alpha_n)$  and  $\beta$  is  $p(\beta_1, \dots, \beta_n)$  then
21:          E ← E ∪ { $\alpha_1 = \beta_1, \dots, \alpha_n = \beta_n$ }
22:        else
23:          return 0 #구조가 일치하지 않으면 실패
24:    return S

```

Figure : Unification algorithm

❖ Example

- ❖ $\text{Unify}(p(x,y,y) \ p(A,z,B))$
- ❖ E is $\{p(x,y,y)=p(A,z,B)\}$.
- ❖ *The first time through the while loop, E becomes $\{x=A, y=z, y=B\}$.
Suppose $x=A$ is selected next.
Then S becomes $\{A/x\}$ and E becomes $\{y=z, y=B\}$.*
- ❖ *Suppose $y=z$ is selected.*
- ❖ *Then y is replaced by z in S and E .*
- ❖ *S becomes $\{A/x, z/y\}$ and E becomes $\{z=B\}$.*
- ❖ *Finally $z=B$ is selected, z is replaced by B , S becomes $\{A/x, B/y, B/z\}$,
and E becomes empty.*
- ❖ *The substitution $\{A/x, B/y, B/z\}$ is returned as an MGU*

- ❖ *Consider unifying $p(A,y,y)$ with $p(z,z,B)$.*
- ❖ *E starts off as $\{p(A,y,y)=p(z,z,B)\}$.*
- ❖ *In the next step, E becomes $\{A=z, y=z, y=B\}$. Then z is replaced by A in E, and E becomes $\{y=A, y=B\}$.*
- ❖ *Then y is replaced by A in E, and E becomes $\{A=B\}$, and then 0 is returned indicating that there is no unifier.*

- 실습) 앞의 unify 알고리즘으로 mgu 구하기

리터럴 집합	mgu	가장 일반적인 치환사례
$\{P(x), P(A)\}$	$\{A/x\}$	$P(A)$
$\{P[f(x), y, g(y)], P[f(x), z, g(x)]\}$	$\{x/y, x/z\}$	$P[f(x), x, g(x)]$
$\{P[f(x, g(A, y)), g(A, y)], P[f(x, z), z]\}$	$\{g(A, y)/z\}$	$P[f(x, g(A, y)), g(A, y)]$

1: $P[f(x), y, g(y)], P[f(x), z, g(x)]$, 2: $S=\{\}$, $E=\{f(x)=f(x), y=z, g(y)=g(x)\}$, 3: $S=\{\}$, $E=\{y=z, g(y)=g(x)\}$, 4: $S=\{z/y\}$, $E=\{g(z)=g(x)\}$, 5: $S=\{z/y\}$, $E=\{z=x\}$, 6: $S=\{x/y, x/z\}$, $E=\{\}$

1: $P[f(x, g(A, y)), g(A, y)] = P[f(x, z), z]$, 2: $E=\{f(x, g(A, y)) = f(x, z), g(A, y)=z\}$, 3: $E=\{x=x, g(A, y)=z, g(A, y)=z\}$, 4: $E=\{g(A, y)=z\}$, 5: $S=\{g(A, y)/z\}$, $E=\{\}$

❖ 다음 각 두 술어들을 단일화한 결과를 쓰시오. 여기에서 B는 상수 기호이고, x와 y는 변수이고, f와 g는 함수 기호이다.

$$1. P(x, f(x), z) \quad \neg P(g(y), f(g(B)), y)$$

$$2. P(x, f(x)) \quad \neg P(f(y), y)$$

논리융합을 사용하려면

- ❖ 논리융합 기법으로 증명하려면 모든 논리식들을 논리곱 표준형(CNF: conjunctive normal form)으로 바꾸어야 한다.
- ❖ 모든 전체 한정사는 없애버린다(기본적으로 가정된다).
- ❖ 존재 한정사는 스코렘 함수로 바꾼다.

술어 논리의 추론

❖ 술어 논리식의 CNF로의 변환 과정

1. 전체 한정사와 존재 한정사를 논리식의 맨 앞으로 끌어내는 변환
2. 전체 한정사에 결합된 변수
 - 임의의 값 허용
3. 존재 한정사에 결합된 변수
 - 대응되는 술어 기호를 참(T)으로 만드는 값을 변수에 대응시킴
 - **스콜렘 함수(Skolem function)**
 - 존재 한정사에 결합된 변수를 해당 술어의 전체 한정사에 결합된 다른 변수들의 새로운 함수로 대체
 - $\forall x \exists y [P(x) \wedge Q(x, y)] \Rightarrow \forall x [P(x) \wedge Q(x, s(x))]$
 - $s(x)$
 - » $Q(x, s(x))$ 를 어떤 x 에 대해서도 참으로 만드는 마법의 함수(magic function)

임의의 정형식을 절 형태로 변경하기

❖ 절 형태(clausal form)로의 변경 단계

1. 함의(implication) 부호를 제거
2. 이중 부정 제거
3. 변수표준화: 각 한정사가 한정하는 변수의 이름이 유일하도록 변수명을 바꿈
4. 존재한정사를 제거한다. (Skolem 함수이용)
5. prenex 형식으로 변경 : 모든 전체한정사를 정형식의 제일 앞으로 이동
6. CNF 정규형으로 변형
7. 전체한정사를 제거
8. 논리곱($\wedge$) 심볼을 제거 : $(\omega_1 \wedge \omega_2)$ 형식의 식을 정형식의 집합 $\{\omega_1, \omega_2\}$ 로 대체
9. 절마다 변수명을 변경

CNF로 변환하기 예

- ① 모든 강아지는 포유류이다.
- ② 바독이는 강아지이다.
- ③ 바독이는 포유류이다.
- ④ 모든 포유류는 우유를 생산한다.



술어논리로 변환

- ① $\forall x (DOG(x) \rightarrow MAMMAL(x)).$
- ② $DOG(badook).$
- ③ $MAMMAL(badook).$
- ④ $\forall x (MAMMAL(x) \rightarrow MILK(x)).$



CNF로 변환

- ① $\neg DOG(x) \vee MAMMAL(x)$
- ② $DOG(badook)$
- ③ $MAMMAL(badook)$
- ④ $\neg MAMMAL(y) \vee MILK(y)$

술어논리 논리융합

❖ Resolution in predicate logic

- γ_1 과 γ_2 : 리터럴의 집합으로 표현된 절
 γ_1 은 아톰 ϕ 포함, γ_2 는 리터럴 $\neg\psi$ 포함
 ϕ 와 ψ 가 mgu μ 를 가지고 있다면
 - 이 두 절은 다음과 같은 논리융합식 ρ 를 가짐

$$\rho = [(\gamma_1 - \{\phi\}) \cup (\gamma_2 - \{\neg\psi\})]\mu$$

예) 다음 두 절을 논리융합하려면

- $P(x) \vee Q(f(x))$
- $R(g(x)) \vee \neg Q(f(A))$
- 먼저 두 번째 절을 $R(g(y)) \vee \neg Q(f(A))$ 로 변경
- 논리융합을 수행하면 $P(A) \vee R(g(y))$ 를 얻음

❖ 예2

- $\{P(x), Q(x, y)\}$ 와 $\{\neg P(A), R(B, z)\}$ 를 논리융합하면 $\{Q(A, y), R(B, z)\}$

❖ 예3

- $\{P(x, x), Q(x), R(x)\}$ 와 $\{\neg P(A, z), \neg Q(B)\}$ 를 논리융합하면
- $\{Q(A), R(A), \neg Q(B)\}$ 또는 $\{P(B, B), R(B), \neg P(A, z)\}$

술어 논리의 추론

❖ 술어 논리로 지식의 증명

- (a) $\forall x[CanRead(x) \rightarrow Literate(x)]$
- (b) $\forall x[Monkey(x) \rightarrow \neg Literate(x)]$
- (c) $\exists x[Monkey(x) \wedge Intelligent(x)]$
- (d) $\exists x[Intelligent(x) \wedge \neg CanRead(x)]$ ← 증명

▪ 논리합 형태로 변환

- (1) $\neg CanRead(x) \vee Literate(x)$
- (2) $\neg Monkey(x) \vee \neg Literate(x)$ x를 y로 변경
- (3) $Monkey(A)$
- (4) $Intelligent(A)$
- (5) $\neg Intelligent(x) \vee CanRead(x)$ x를 z로 변경



(d)를 논리융합 반박을 이용하여 증명하기 부정

▪ (1)과 (5)

$$(6) \neg Intelligent(x) \vee Literate(x)$$

▪ (2)와 (6)

$$(7) \neg Intelligent(x) \vee \neg Monkey(x)$$

▪ (3)과 (7)

$$(8) \neg Intelligent(A)$$

▪ (4)와 (8)

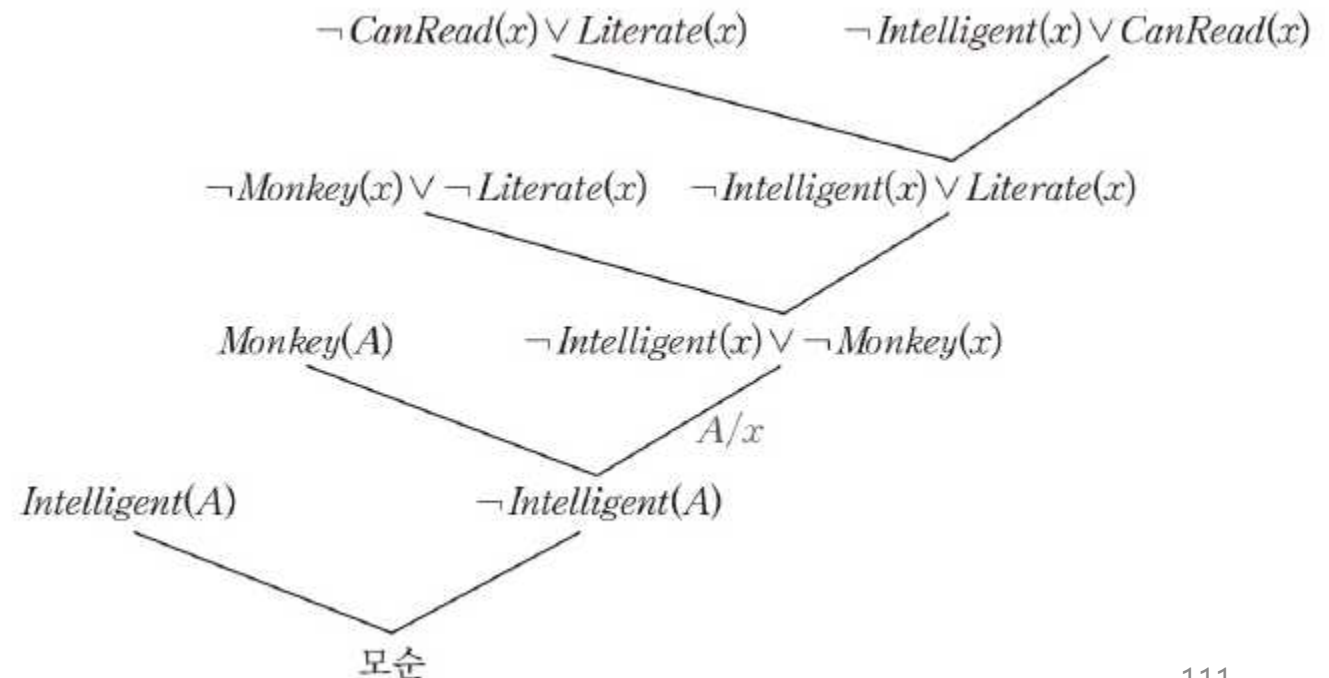
$$(9) Intelligent(A) \wedge \neg Intelligent(A) \equiv nil$$

(d)를 부정하여 모순이 발생하므로, (d)가 참임

술어 논리의 추론

❖ 술어 논리로 지식의 증명

- (1) $\neg CanRead(x) \vee Literate(x)$
- (2) $\neg Monkey(x) \vee \neg Literate(x)$
- (3) $Monkey(A)$
- (4) $Intelligent(A)$
- (5) $\neg Intelligent(x) \vee CanRead(x)$



논리융합을 이용한 정리증명

❖ 정리증명 단계

- Δ 에 있는 모든 정형식을 절 형태로 변형
- 증명하고자 하는 정형식 ω 를 부정(negate)한 절을 Δ 에 대한 절에 추가
- 공절이 추론될 때까지 논리융합을 적용

❖ 예제

- "모든 사람은 죽는다"와 "소크라테스는 사람이다"로부터 "소크라테스는 죽는다"를 추론
- $\Delta = \{(\forall x)[\text{Man}(x) \supset \text{Mortal}(x)], \text{Man}(\text{Socrates})\}$
- $\omega = \text{Mortal}(\text{Socrates})$

1. 절 형태로 변형 : $\{\neg \text{Man}(x) \vee \text{Mortal}(x), \text{Man}(\text{Socrates})\}$
2. ω 의 부정을 추가: $\neg \text{Mortal}(\text{Socrates})$
3. 논리융합을 적용
 1. $\neg \text{Man}(x) \vee \text{Mortal}(x)$
 2. $\text{Man}(\text{Socrates})$
 3. $\neg \text{Mortal}(\text{Socrates})$
 4. $\neg \text{Man}(\text{Socrates})$ (1,3을 논리융합 - $mgu = \{\text{Socrates}/x\}$)
 5. NIL (2,4를 논리융합)

논리융합에 의한 증명

1. 증명하고자 하는 사실을 부정하여 절들의 리스트에 추가한다.
2. 지식 베이스의 문장들을 CNF 형태로 변환한다.
3. 논리융합할 수 있는 절의 쌍이 더 이상 없을 때까지 다음을 반복한다.
 - 3.1. 논리융합할 수 있는 절의 쌍을 찾아 논리융합한다.
 - 3.2. 논리융합절을 절들의 리스트에 추가한다.
 - 3.3. NIL이 유도되면, 증명하고자 하는 사실이 참이다.
4. 증명하고자 하는 사실의 부정이 거짓이다.

논리융합에 의한 증명의 예

- ① $\neg \text{DOG}(x) \vee \text{MAMMAL}(x)$
- ② $\text{DOG}(\text{badook})$
- ③ $\text{MAMMAL}(\text{badook})$
- ④ $\neg \text{MAMMAL}(y) \vee \text{MILK}(y)$

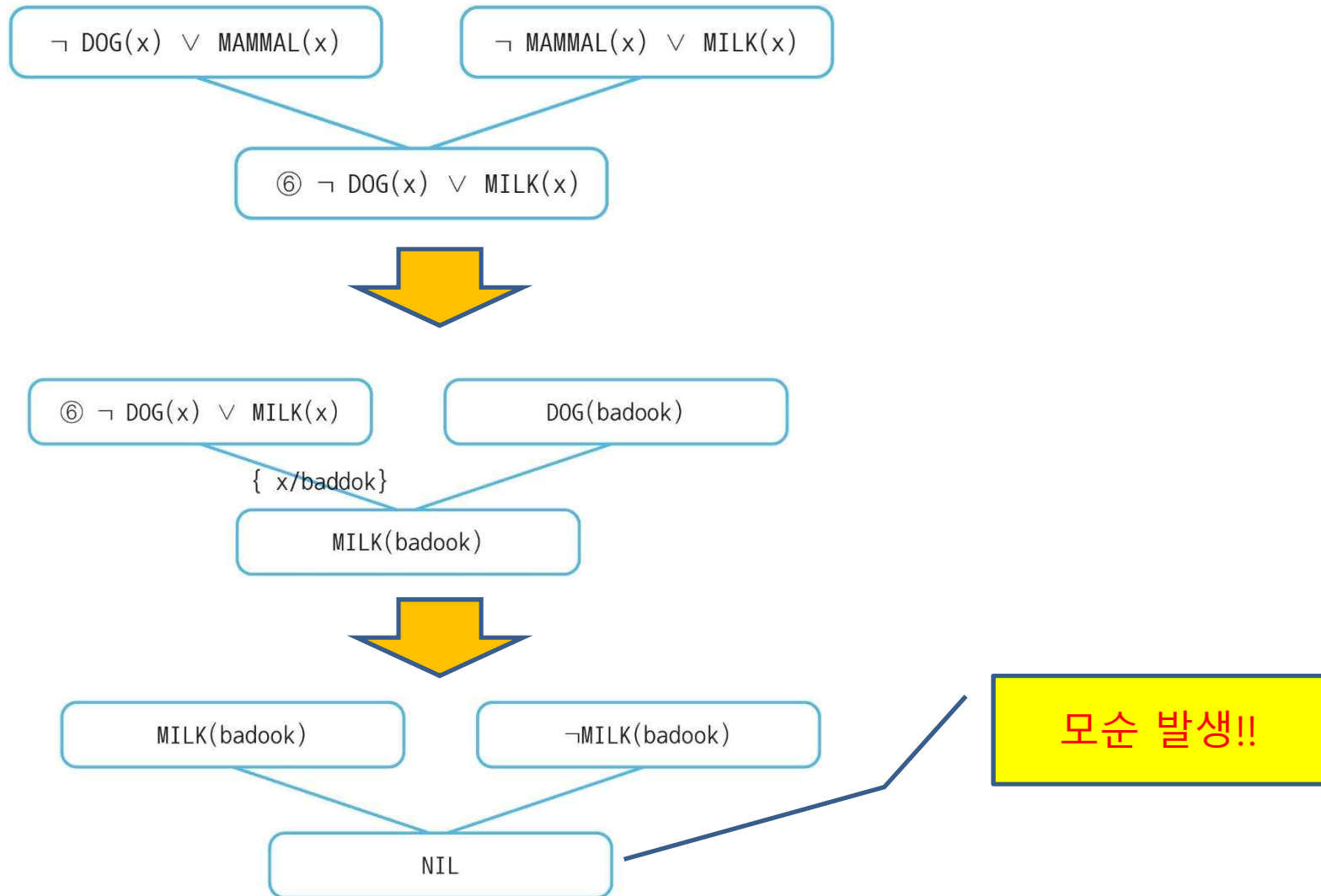
증명하고 싶은 사실

MILK(badook)



- ① $\neg \text{DOG}(x) \vee \text{MAMMAL}(x)$
- ② $\text{DOG}(\text{badook})$
- ③ $\text{MAMMAL}(\text{badook})$
- ④ $\neg \text{MAMMAL}(y) \vee \text{MILK}(y)$
- ⑤ $\neg \text{MILK}(\text{badook})$

논리융합에 의한 증명의 예



Lab: 논리융합 연습 #1

- ❖ 간단한 논리융합 연습을 해보자. 다음과 같은 지식이 있다.
 - ① 어떤 것이 지능적이라면 그것은 상식을 가져야 한다.
 - ② PC는 상식이 없다.
- ❖ 위의 지식을 바탕으로 'PC는 지능적이지 않다'를 증명해 보자.

Lab: 논리융합 연습

먼저 지식들을 술어 논리로 표현한다.

① $\forall x(I(x) \Rightarrow CS(x))$

② $\neg CS(PC)$

일단 위의 술어 논리식을 CNF 형식으로 변환한다.

① $\neg I(x) \vee CS(x)$

② $\neg CS(PC)$

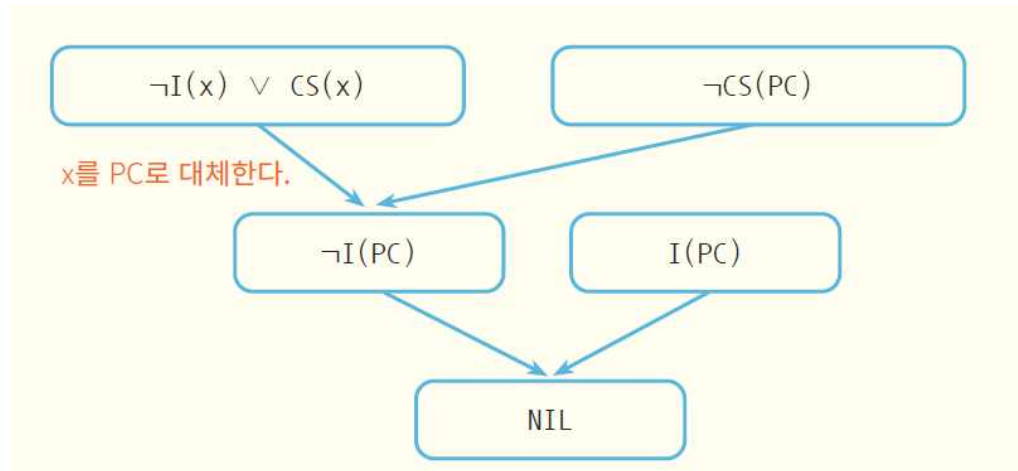
우리가 증명하고자 하는 것 $\neg I(PC)$ 을 반대로 하여 $I(PC)$ 을 지식에 추가한다.

① $\neg I(x) \vee CS(x)$

② $\neg CS(PC)$

③ $I(PC)$

Lab: 논리융합 연습



Lab: 논리융합 연습 #2

① $\text{Father}(\text{Kim}, \text{Kim2})$

② $\text{Alive}(\text{Kim})$

③ $\forall x \forall y \text{ Father}(x, y) \Rightarrow \text{Parent}(x, y)$

④ $\forall x \forall y (\text{Parent}(x, y) \wedge \text{Alive}(x)) \Rightarrow \text{Older}(x, y)$

위의 지식을 바탕으로 $\text{Older}(\text{Kim}, \text{Kim2})$ 를 증명해 보자.

Lab: 논리융합 연습 #2

① $\text{Father}(\text{Kim}, \text{Kim2})$

② $\text{Alive}(\text{Kim})$

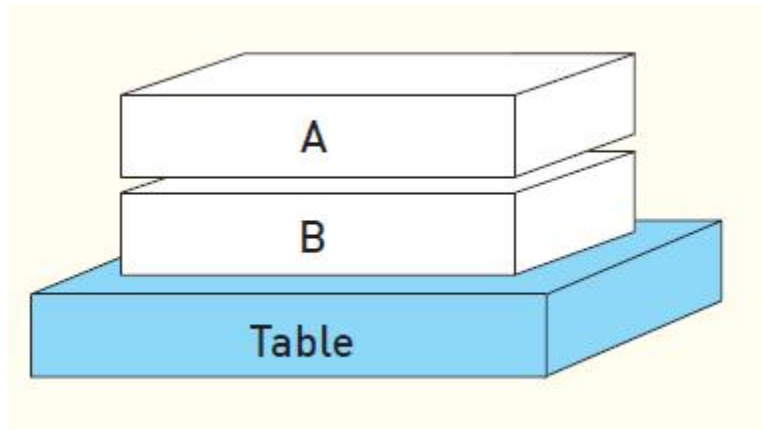
③ $\neg \text{Father}(x, y) \vee \text{Parent}(x, y)$

④ $\neg \text{Parent}(x, y) \vee \neg \text{Alive}(x) \vee \text{Older}(x, y)$

⑤ $\neg \text{Older}(\text{Kim}, \text{Kim2})$

Lab: 논리융합 연습 #3

- ❖ 다음과 같은 상황을 술어 논리를 이용하여 표현하고 $ABOVE(A, Table)$ 을 증명해 보자.



Lab: 논리융합 연습 #3

- ❖ 테이블 위에 블록 B가 놓여져 있고 블록 B 위에 블록 A가 놓여져 있다. 따라서 우리가 관찰한 사실은 다음과 같이 술어 논리로 표현할 수 있다.

ON(A, B)

ON(B, Table)

- ❖ 우리는 상식으로 다음과 같은 사실을 알고 있다.

$\forall x \forall y [ON(x, y) \rightarrow ABOVE(x, y)]$

$\forall x \forall y \forall z [ABOVE(x, y) \wedge ABOVE(y, z) \rightarrow ABOVE(x, z)]$

Lab: 논리융합 연습 #3 실습

- ❖ 일단 위의 논리식을 CNF 형식으로 변환하여야 한다. 함의의 등가식을 이용하여 다음과 같이 변환할 수 있다.

$$\neg \text{ON}(x, y) \vee \text{ABOVE}(x, y)$$

$$\neg \text{ABOVE}(a, b) \vee \neg \text{ABOVE}(b, c) \vee \text{ABOVE}(a, c)$$

여기에 우리가 증명하려고 하는 사실을 부정하여 추가한 후에 이미 알고 있는 사실을 합치면 다음과 같이 된다.

① $\neg \text{ON}(x, y) \vee \text{ABOVE}(x, y)$

② $\neg \text{ABOVE}(a, b) \vee \neg \text{ABOVE}(b, c) \vee \text{ABOVE}(a, c)$

③ $\text{ON}(A, B)$

④ $\text{ON}(B, \text{Table})$

⑤ $\neg \text{ABOVE}(A, \text{Table})$

⑥(1, 3) $\text{ABOVE}(A, B)$, ⑦(1,4) $\text{ABOVE}(B, \text{Table})$,

⑧(2 가운데로,6) $\neg \text{ABOVE}(a, A) \vee \text{ABOVE}(a, B)$,

⑨(2 가운데로 7) $\neg \text{ABOVE}(a, B) \vee \text{ABOVE}(a, \text{Table})$

⑩(5,9) $\neg \text{ABOVE}(A, B)$

⑪(6, 10) NIL

- ❖ '모든 말은 개보다 빠르다. 모든 토끼보다 빠른 그레이하운드 개가 있다. 조랑이는 말이다. 쫓긋이는 토끼다'라는 것을 알고 있을 때, '조랑이'가 '쫓긋이'보다 빠르다는 것을 증명하시오. 편의상 다음과 같이 술어 논리식으로 위 내용이 표현되어있다고 하자.

$$\forall x \forall y (Horse(x) \wedge Dog(y)) \rightarrow Faster(x, y)$$

$$\exists y Greyhound(y) \wedge (\forall z Rabbit(z) \rightarrow Faster(y, z))$$

$Horse$ (조랑이)

$Rabbit$ (쫓긋이)

$$\forall y Greyhound(y) \rightarrow Dog(y)$$

$$\forall x \forall y \forall z (Faster(x, y) \wedge Faster(y, z)) \rightarrow Faster(x, z)$$

$$\neg(Horse(x) \wedge Dog(y)) \vee Faster(x, y)$$

$$Greyhound(백구) \wedge (\neg Rabbit(z) \vee Faster(백구, z))$$

$Horse$ (조랑이)

$Rabbit$ (쫓긋이)

$$\neg Greyhound(y) \vee Dog(y)$$

$$\neg(Faster(x, y) \wedge Faster(y, z)) \vee Faster(x, z)$$

$Faster(조랑이, 쫓긋이)$ <- 증명

$$\begin{aligned}
& \neg(Horse(x) \wedge Dog(y)) \vee Faster(x, y) \\
1: & \neg Horse(x) \vee \neg Dog(y) \vee Faster(x, y) \\
& Greyhound(백구) \wedge (\neg Rabbt(z) \vee Faster(백구, z)) \\
2: & Greyhound(백구) \\
3: & \neg Rabbt(z) \vee Faster(백구, z) \\
4: & Horse(조랑이) \\
5: & Rabbt(종긋이) \\
6: & \neg Greyhound(a) \vee Dog(a) \\
& \neg(Faster(x, y) \wedge Faster(y, z)) \vee Faster(x, z) \\
7: & \neg Faster(b, c) \vee \neg Faster(c, d) \vee Faster(b, d) \\
8: & \neg Faster(조랑이, 종긋이) \\
9(1,8): & \neg Horse(조랑이) \vee \neg Dog(종긋이) \\
10(9,6): & \neg Greyhound(종긋이) \vee \neg Horse(조랑이) \\
11(10,4): & \neg Greyhound(종긋이) \\
12(7,8): & \neg Faster(조랑이, c) \vee \neg Faster(c, 종긋이) \\
13(3, 12) \{백구/c, 종긋이/z\}: & \neg Rabbt(종긋이) \vee \neg Faster(조랑이, 백구) \\
14(1,13): & \neg Horse(조랑이) \vee \neg Dog(백구) \vee \neg Rabbt(종긋이) \\
15(5, 14): & \neg Horse(조랑이) \vee \neg Dog(백구) \\
16(6, 15)\{백구/a\}: & \neg Greyhound(백구) \vee \neg Horse(조랑이) \\
17(2, 16): & \neg Horse(조랑이) \\
18(4, 17): & Nil(F)
\end{aligned}$$

답변추출

❖ 답변 리터럴(answer literal)

- 반박 과정 중에 적용된 치환을 기억
- 증명하고자 하는 정리에 대한 부정으로부터 만들어진 각 절에 $\text{Ans}(\xi_1, \xi_2, \dots)$ 라는 리터럴을 추가하고 하나의 답변 리터럴이 남을 때까지 논리융합을 수행

❖ “모든 사람은 죽는다”와 “소크라테스는 사람이다”로부터 “누가 죽는가?”를 추론

- $\Delta = \{(\forall x)[\text{Man}(x) \supset \text{Mortal}(x)], \text{Man}(\text{Socrates})\}$
- $\omega = \text{Mortal}(y)$

1. $\neg \text{Man}(x) \vee \text{Mortal}(x)$
2. $\text{Man}(\text{Socrates})$
3. $\neg \text{Mortal}(y) \vee \text{Ans}(y)$
4. $\text{Mortal}(\text{Socrates})$ (1, 2를 논리융합 - $mgu = \{\text{Socrates}/x\}$)
5. $\text{Ans}(\text{Socrates})$ (3,4를 논리융합 - $mgu = \{\text{Socrates}/y\}$)

논리 프로그래밍 언어

❖ Horn 절 (Horn clause)

- 논리식을 논리합의 형태로 표현할 때, $\neg A(x) \vee \neg B(x) \vee C(x)$ 와 같이 긍정인 리터럴을 최대 하나만 허용

❖ Prolog

- Horn 절만 허용하는 논리 프로그래밍 언어
 - Prolog의 규칙에서는 논리곱만을 사용, 변수는 대문자

father(noah, shem).

father(noah, ham).

father(shem, elam).

father(shem, arphaxad).

father(arphaxad, caina).

grandfather(X,Y) :- father(X,Z), father(Z,Y).

:- grandfather(X,Y).

- X, Y 값 출력

father(X,Z) $\wedge$ father(Z,Y) $\rightarrow$ grandfather(X,Y)

- 백트래킹(backtracking)을 이용하여 실행

프롤로그(Prolog)

- ❖ <https://www.swi-prolog.org/>에서 다운로드할 수 있는 SWI-Prolog를 설치해보자.

```
parent(Y,X) :- child(X,Y).  
father(Y,X) :- child(X,Y), male(Y).
```

- ❖ 프롤로그에서는 변수를 항상 대문자로 작성해야 한다.
- ❖ “father(Y,X) :- child(X,Y), male(Y).”은 X가 Y의 자식이고 Y가 남성이면 Y는 X의 아버지이다. ”를 의미한다. 즉 $\text{child}(X,Y) \wedge \text{male}(Y) \rightarrow \text{father}(Y,X)$ 을 의미한다.
- ❖ 프롤로그의 규칙에서 논리합은 사용하지 않는다. 논리곱만을 사용한다.

프롤로그(Prolog)에서 사실 입력

- ❖ 상수는 소문자로 입력한다.

```
child(son1, kim).  
child(son2, kim).  
male(kim).  
male(son1).  
male(son2).
```

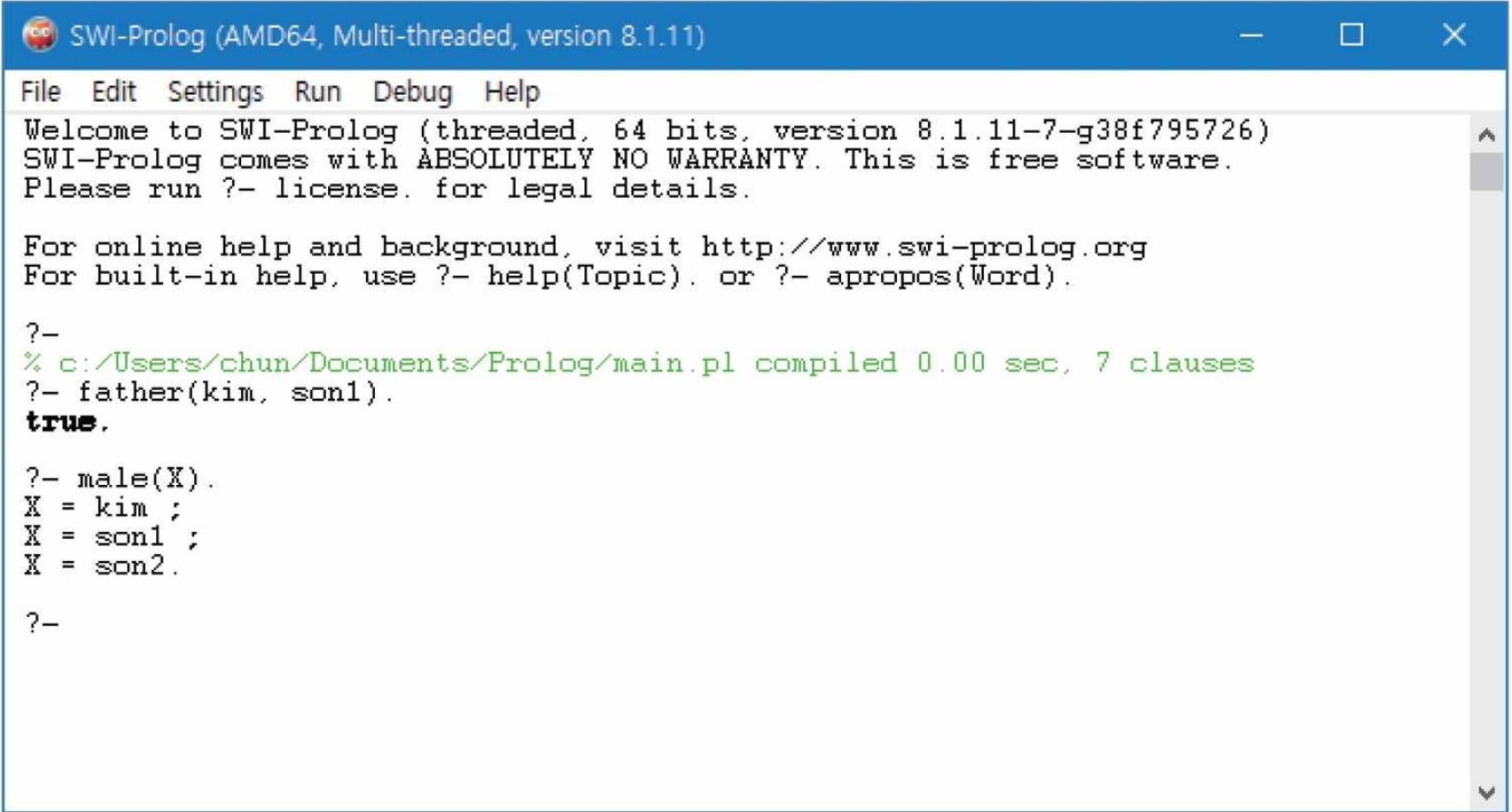
프롤로그 실습

- ① 다음과 같은 문장들을 저장하는 파일 main.pl을 생성한다.

```
child(son1, kim).  
child(son2, kim).  
male(kim).  
male(son1).  
male(son2).  
parent(Y,X) :- child(X,Y).  
father(Y,X) :- child(X,Y), male(Y).
```

- ② [File]->[Consult...] 메뉴 항목을 선택하여서 main.pl을 불러들인다.

프롤로그 실습



```
SWI-Prolog (AMD64, Multi-threaded, version 8.1.11)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 8.1.11-7-g38f795726)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit http://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
% c:/Users/chun/Documents/Prolog/main.pl compiled 0.00 sec, 7 clauses
?- father(kim, son1).
true.

?- male(X).
X = kim ;
X = son1 ;
X = son2.

?-
```

Summary

134

- 많이 사용되는 지식 표현의 방법은 “규칙”, “의미망”, “프레임”, “술어 논리” 등이 있다.
- 명제 논리에서 추론 방법은 “모더스 포넨스”, “부정 논법”, “삼단논법” 등이 있다.
- 의미망(semantic network)은 방향 그래프를 이용하여 개념 간의 관계를 나타내는 방법이다.
- 프레임(frame)은 1970년대에 Marvin Minsky가 제안한 지식 표현 방법으로 특정 객체와 그 속성을 묶어서 하나로 조직화하는 방법이다.
- 술어 논리(predicate logic)는 하나의 명제가 객체(object, 또는 인수)와 술어(predicate)로 나누어질 수 있고 변수와 한정사를 사용할 수 있는 논리이다.
- 술어 논리식에서는 추론을 위하여 추가로 논리융합(resolution)이라는 방법을 사용한다.

Q & A

135

